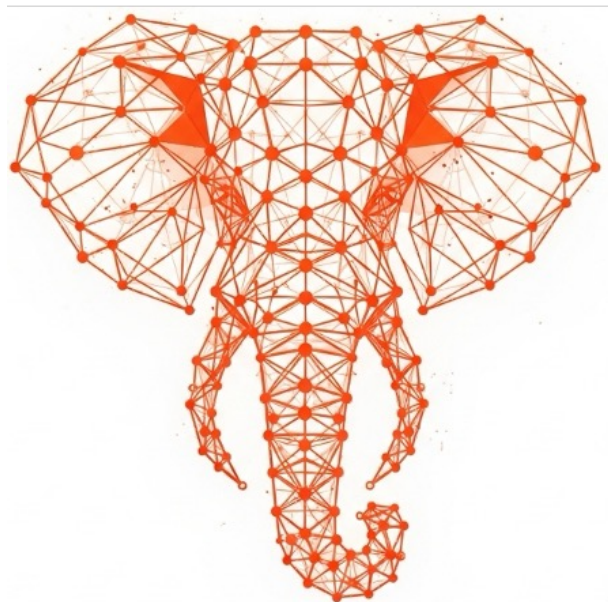




Deep Learning

Advanced Models & Methods

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Laurea Magistrale in Informatica

Dip. di Matematica e Informatica

Università di Catania

Video Understanding

What is Video Understanding?



<https://www.youtube.com/watch?v=PzyQnUdvzdY>

Let's check your Video Understanding Abilities

Who is the protagonist in this scene?

- A) A stage crew member
- B) One of the actors already at the table
- ☒ C) The man who joins the other actors
- D) The cameraman

Who are the main actors in this scene?

- A) Only the protagonist
- ☒ B) The protagonist and other actors at the table
- C) The protagonist and the director
- D) The protagonist and the audience

What is happening in the scene?

- A) The protagonist is giving a speech to the other actors
- ☒ B) The protagonist joins the group of actors who are rehearsing a scene together
- C) The protagonist is performing a solo act
- D) The protagonist is having a discussion with the stage crew

What is the context of the scene?

- ☒ A) The protagonist is preparing for a theater performance
- B) The protagonist is meeting with the producers
- C) The protagonist is taking a break backstage
- D) The protagonist is giving a lecture

Where is the scene set?

- A) A house kitchen
- B) A backstage
- ☒ C) A theatre stage
- D) A film's set

How is the interaction between the characters in this scene?

- A) Confrontational
- ☒ B) Collaborative but a bit tense
- C) Isolated and quiet
- D) Competitive

What is the likely event that will follow in this scene?

- A) The main character will begin his performance on stage
- ☒ B) The actors will continue to rehearse
- C) The main character will leave the room
- D) The actors will have a fight

What did the protagonist do before joining the rehearsal session?

- A) Call his daughter on the phone
- B) Open the fridge and grab some water
- C) Get out of the car
- ☒ D) Talk to a crew member in the backstage

How would you describe the protagonist's mood in this scene?

- A) Excited and eager to perform
- B) Nervous and anxious
- C) Calm and contemplative
- ☒ D) Tired and frustrated

What is Video Understanding?

«analyze video to extract information»

**A single frame
would not be
enough**

Scene

- Backstage
- Theatre stage

Objects

- Table
- Scripts
- Kitchen furniture

Actors

- Protagonist
- Other actors
- Stage crew members



Actions

- Conversation
- Rehearsing

Context

- A rehearsal for the play

Interactions

- A talks to B
- C asks advice to A
- A replies to C

Mood

- Exhaustion
- Frustration
- Tenseness

High Dimensionality and Computational Requirements

Single Image



Video



VS

Size per single image

SD (720 x 480) ~ 1.04 MB

HD (1920 x 1080) ~ 6.22 MB

Size per minute (30fps)

SD (720 x 480) ~ 1.86 GB

HD (1920 x 1080) ~ 10.44 GB

- **Lower-dimensional Data:**
 - One frame, one spatial context
 - Fixed set of objects
 - Easier to analyze in isolation
- **Computational Simplicity:**
 - Single inference step
 - Less memory and compute
- **Typical tasks:**
 - Object Detection
 - Image Classification
 - Segmentation
- **Limitations:**
 - Does not model motion, interactions, and dynamic events

- **High-dimensional Data:**
 - Multiple frames over time
 - Time greatly increases data (frames × pixels × channels) with no commensurate increase of information (a lot of redundancy!)
- **Computational Complexity:**
 - Requires analyzing sequences, memory mechanisms (3D CNNs, LSTMs, Transformers)
 - Higher computational and memory demands
 - Real-time inference adds additional complexity
- **Typical Tasks:**
 - Action Recognition
 - Temporal Action Detection
 - Event Understanding and Prediction

Modeling Temporal Dependencies

Videos are characterized by short-/medium-/long-range dependencies, which are critical to model to obtain effective video understanding.

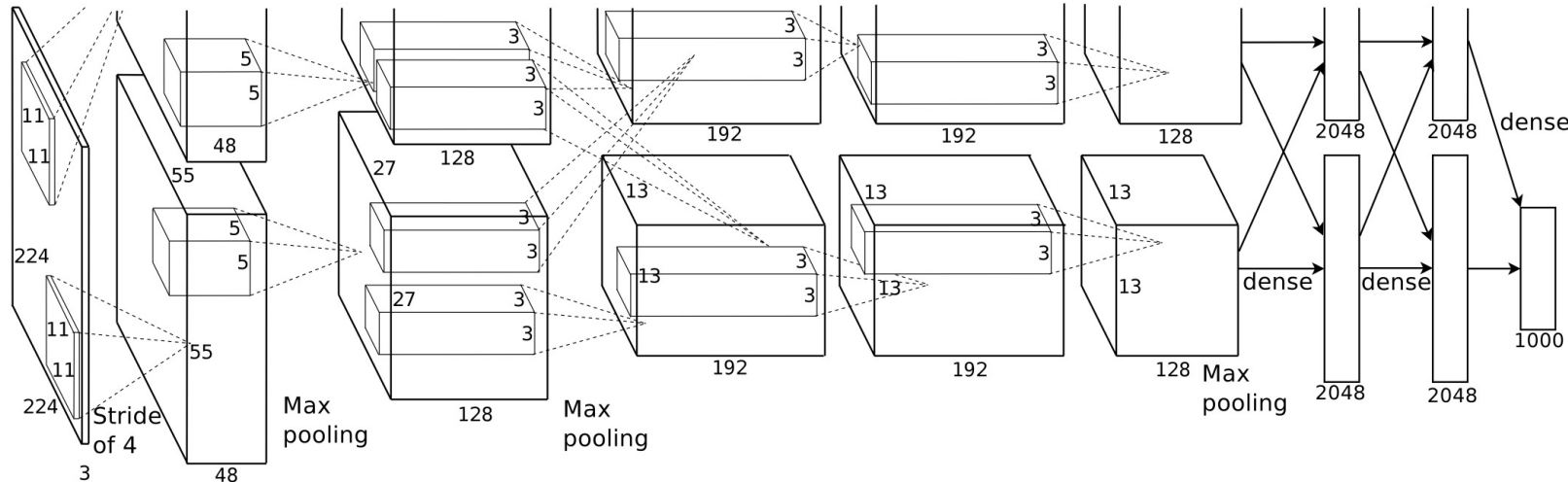
These relationships are characterized by different granularities and hence they are not trivial to model.

long range – same actor



What's the single most influential task in deep learning?

Image Classification & Representation learning



- The image classification task has been instrumental for learning good representations
- Consider in particular ImageNet and AlexNet
- Deep models trained on large-scale image classification tasks *transfer* to other tasks and datasets
- Video is no different!

A. Krizhevsky, I. Sutskever, e G. E. Hinton, «ImageNet Classification with Deep Convolutional Neural Networks», in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., NeurIPS 2012.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, e L. Fei-Fei, «ImageNet: A Large-Scale Hierarchical Image Database», CVPR 2009.

Video Classification



Images: Recognize objects



Dog
Cat
Fish
Truck



Videos: Recognize actions



Swimming
Running
Jumping
Eating
Standing

<https://upload.wikimedia.org/wikipedia/commons/1/17/Running.gif>

Video Classification with Deep Neural Network



<https://upload.wikimedia.org/wikipedia/commons/1/17/Running.gif>

Deep Neural Network

Swimming
Running
Jumping
Eating
Standing

Challenge: Video is High-Dimensional!

recall...

Single Image

Size per single image

SD (720 x 480) ~ 1.04 MB

HD (1920 x 1080) ~ 6.22 MB



VS

Video



Size per minute (30fps)

SD (720 x 480) ~ 1.86 GB

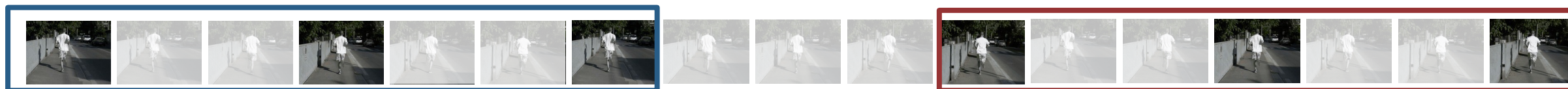
HD (1920 x 1080) ~ 10.44 GB

Raw video: Long, high FPS

Solution: Train on short **clips**: low fps and low spatial resolution.
E.g. $T = 16$, $H=W=112$ (3.2 seconds at 5 fps, 588 KB)



Training: Train model to classify short clips with low FPS



Testing: Run model on different clips, average predictions



Large-scale Video Classification with CNNs

A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, e L. Fei-Fei, «Large-Scale Video Classification with Convolutional Neural Networks», in *IEEE Conference on Computer Vision and Pattern Recognition*, Columbus, OH, USA: IEEE, 2014, pp. 1725–1732.

Large-scale Video Classification with Convolutional Neural Networks

Andrej Karpathy^{1,2}

karpathy@cs.stanford.edu

Thomas Leung¹

leungt@google.com

George Toderici¹

gtoderici@google.com

Rahul Sukthankar¹

sukthankar@google.com

Sanketh Shetty¹

sanketh@google.com

Li Fei-Fei²

feifeili@cs.stanford.edu

¹Google Research

²Computer Science Department, Stanford University

<http://cs.stanford.edu/people/karpathy/deepvideo>

Abstract

Convolutional Neural Networks (CNNs) have been established as a powerful class of models for image recognition problems. Encouraged by these results, we provide an extensive empirical evaluation of CNNs on large-scale video classification using a new dataset of 1 million YouTube videos belonging to 487 classes. We study multiple approaches for extending the connectivity of a CNN in time domain to take advantage of local spatio-temporal information and suggest a multiresolution, foveated architecture as a promising way of speeding up the training. Our best spatio-temporal networks display significant performance improvements compared to strong feature-based baselines (55.3% to 63.9%), but only a surprisingly modest improvement compared to single-frame models (59.3% to 60.9%). We further study the generalization performance of our best model by retraining the top layers on the UCF-101 Action Recognition dataset and observe significant performance improvements compared to the UCF-101 baseline model (63.3% up from 43.9%).

1. Introduction

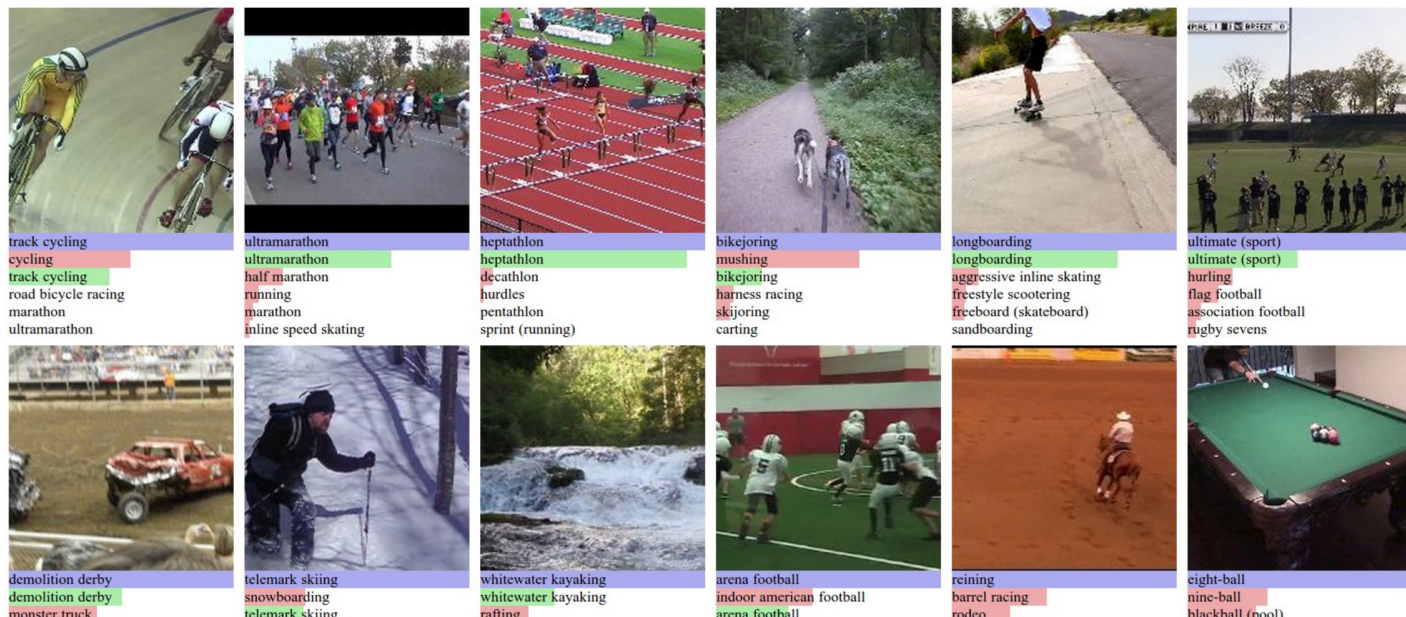
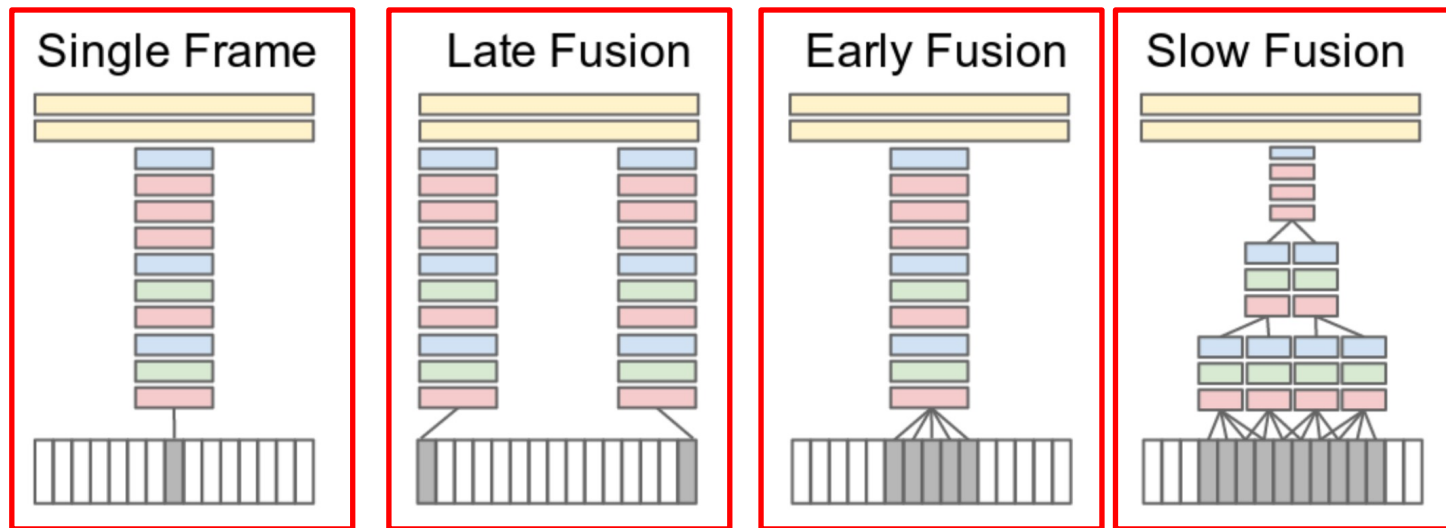
Images and videos have become ubiquitous on the internet, which has encouraged the development of algorithms that can analyze their semantic content for various applications, including search and summarization. Recently, Convolutional Neural Networks (CNNs) [15] have been demonstrated as an effective class of models for understanding image content, giving state-of-the-art results on image recognition, segmentation, detection and retrieval [11, 3, 2, 20, 9, 18]. The key enabling factors behind these results were techniques for scaling up the networks to tens of millions of parameters and massive labeled datasets that

image features [28]. Encouraged by positive results in domain of images, we study the performance of CNNs in large-scale video classification, where the networks have access to not only the appearance information present in single, static images, but also their complex temporal evolution. There are several challenges to extending and applying CNNs in this setting.

From a practical standpoint, there are currently no video classification benchmarks that match the scale and variety of existing image datasets because videos are significantly more difficult to collect, annotate and store. To obtain sufficient amount of data needed to train our CNN architectures, we collected a new Sports-1M dataset, which consists of 1 million YouTube videos belonging to a taxonomy of 487 classes of sports. We make Sports-1M available to the research community to support future work in this area.

From a modeling perspective, we are interested in answering the following questions: what temporal connectivity pattern in a CNN architecture is best at taking advantage of local motion information present in the video? How does the additional motion information influence the predictions of a CNN and how much does it improve performance overall? We examine these questions empirically by evaluating multiple CNN architectures that each take a different approach to combining information across the time domain.

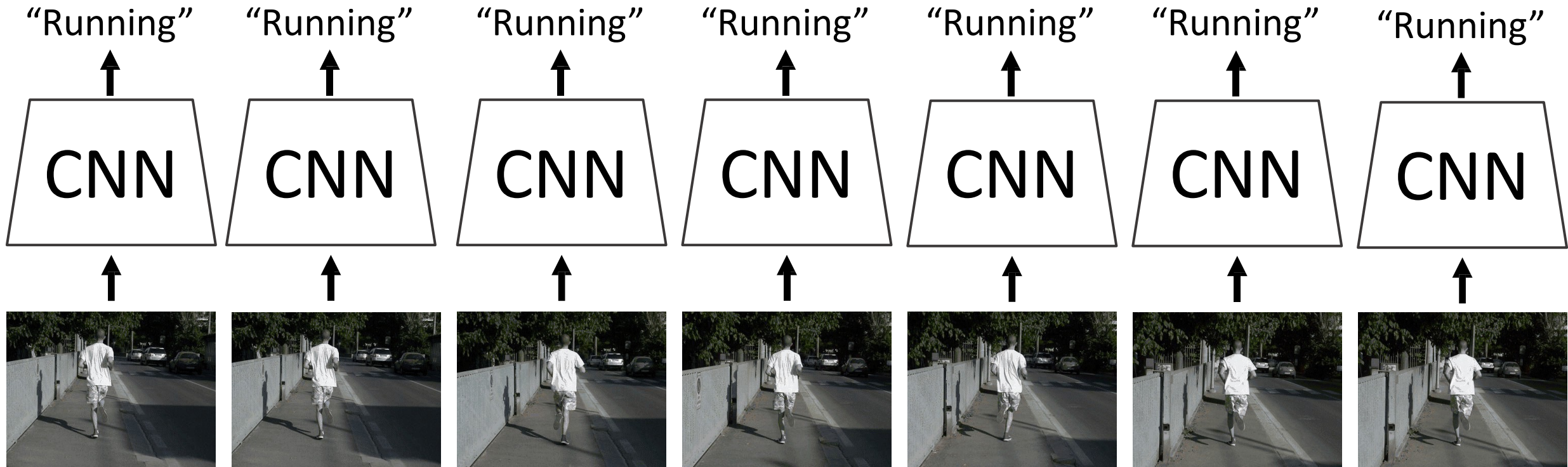
From a computational perspective, CNNs require extensively long periods of training time to effectively optimize the millions of parameters that parametrize the model. This difficulty is further compounded when extending the connectivity of the architecture in time because the network must process not just one image but several frames of video at a time. To mitigate this issue, we show that an effective approach to speeding up the runtime performance of CNNs is to modify the architecture to contain two separate streams of processing: a *context* stream that learns features



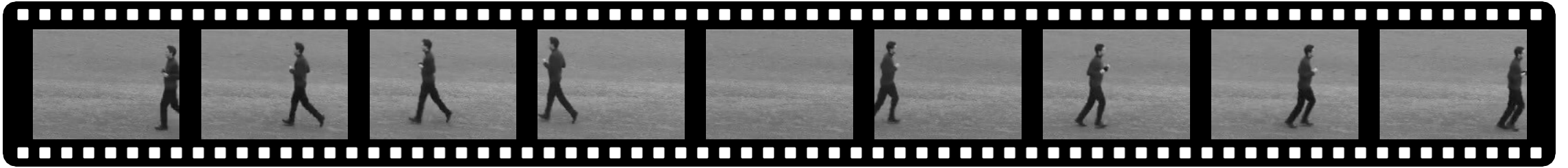
Video Classification: Single-Frame CNN

Simple idea: train normal 2D CNN to classify video frames independently!
(Average predicted probs at test-time)

Often a **very** strong baseline for video classification



Different ways to see video



1) Single Image



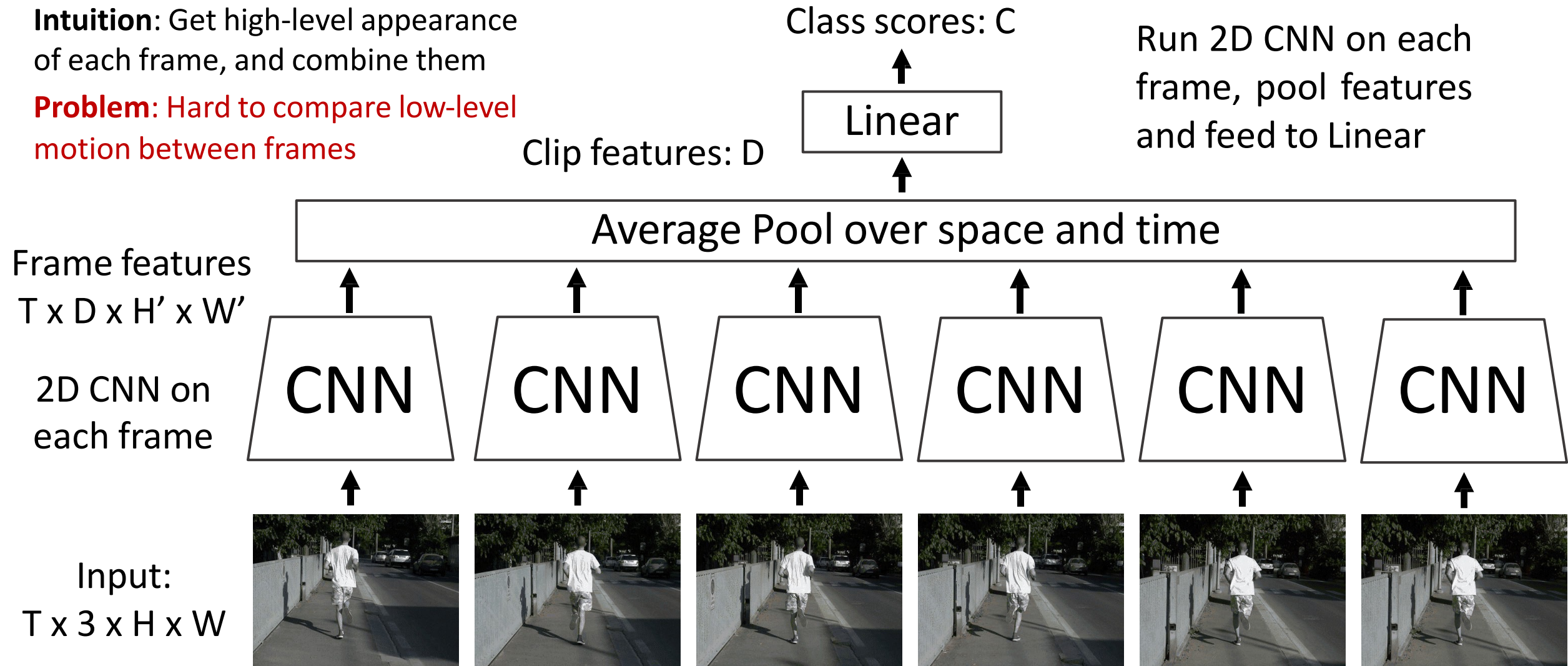
2) A Bag of Images



Video Classification: Late Fusion (with pooling)

Intuition: Get high-level appearance of each frame, and combine them

Problem: Hard to compare low-level motion between frames



Video Classification: Late Fusion (with FC layers)

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

Intuition: Get high-level appearance of each frame, and combine them

Class scores: C

MLP

Run 2D CNN on each frame, concatenate features and feed to MLP

Clip features: $TDH'W'$

Flatten

Frame features
 $T \times D \times H' \times W'$

2D CNN on
each frame

CNN

CNN

CNN

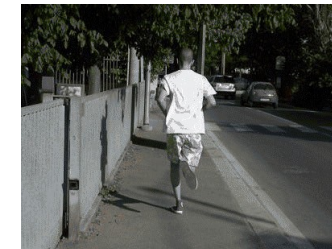
CNN

CNN

CNN

Input:

$T \times 3 \times H \times W$



Video Classification: Early Fusion

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

Intuition: Compare frames with very first conv layer, after that normal 2D CNN

Problem: One layer of temporal processing may not be enough!

First 2D convolution collapses all temporal information:

Input: $3T \times H \times W$

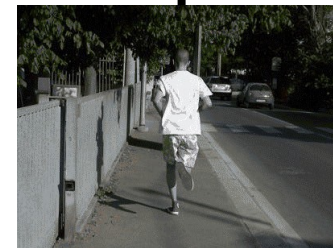
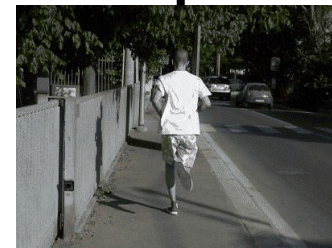
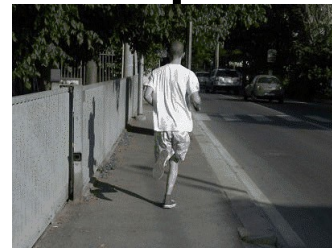
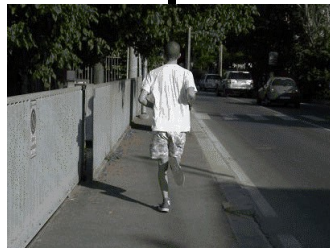
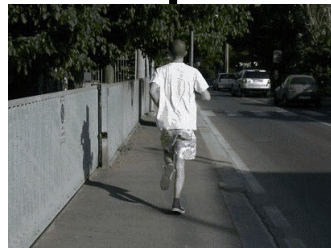
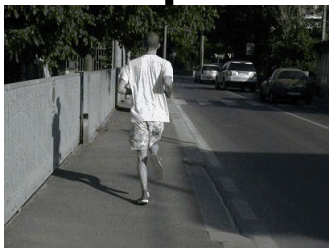
Output: $D \times H \times W$

Reshape:

$3T \times H \times W$

Input:

$T \times 3 \times H \times W$

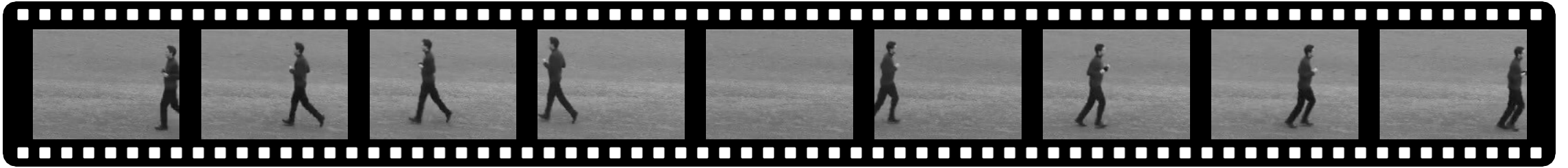


Class scores: C

2D CNN

Rest of the network is standard 2D CNN

Different ways to see video



1) Single Image



2) A Bag of Images



3) 3D (x,y,t) volume

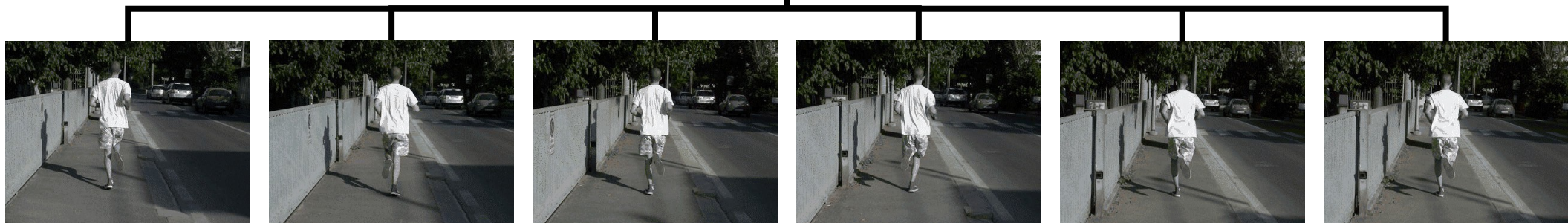
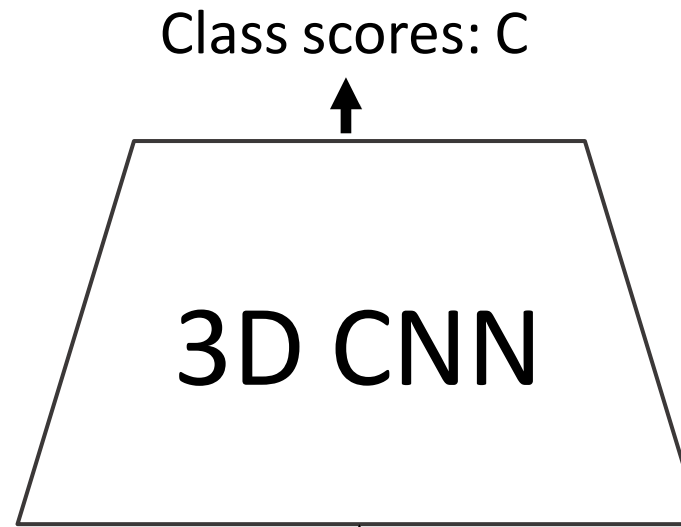


Video Classification: 3D CNN

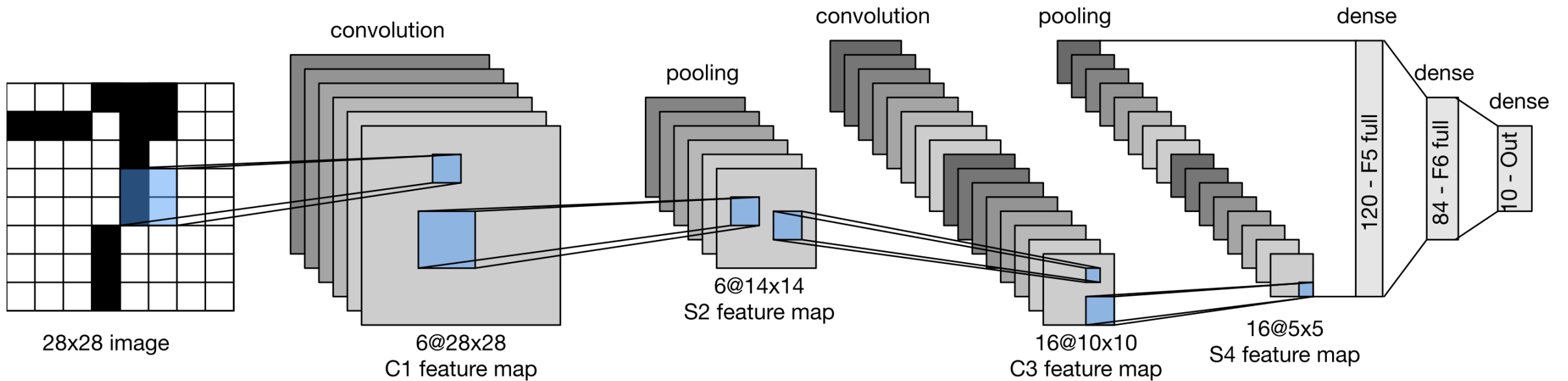
Ji et al, "3D Convolutional Neural Networks for Human Action Recognition", TPAMI 2010 ;
Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

Intuition: Use 3D versions of convolution and pooling to slowly fuse temporal information over the course of the network

Each layer in the network is a 4D tensor: $D \times T \times H \times W$
Use 3D conv and 3D pooling operations



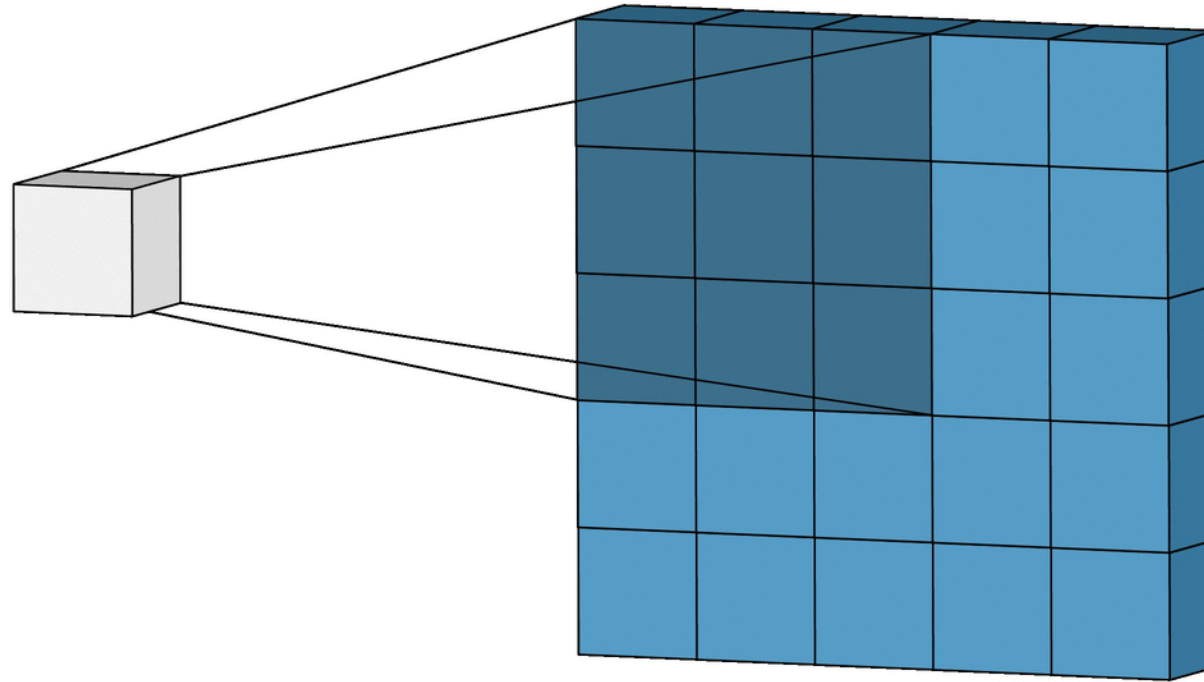
2D Convolution



LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

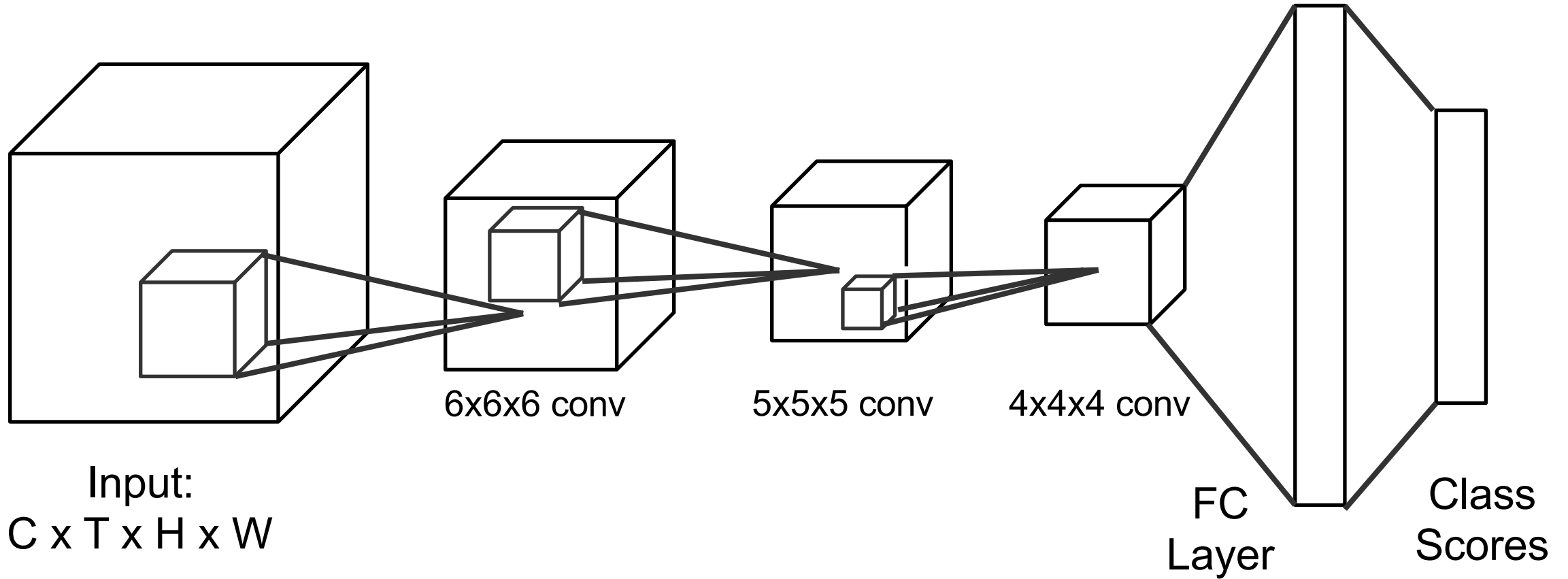
<https://en.wikipedia.org/wiki/LeNet>

2D Convolution Animation

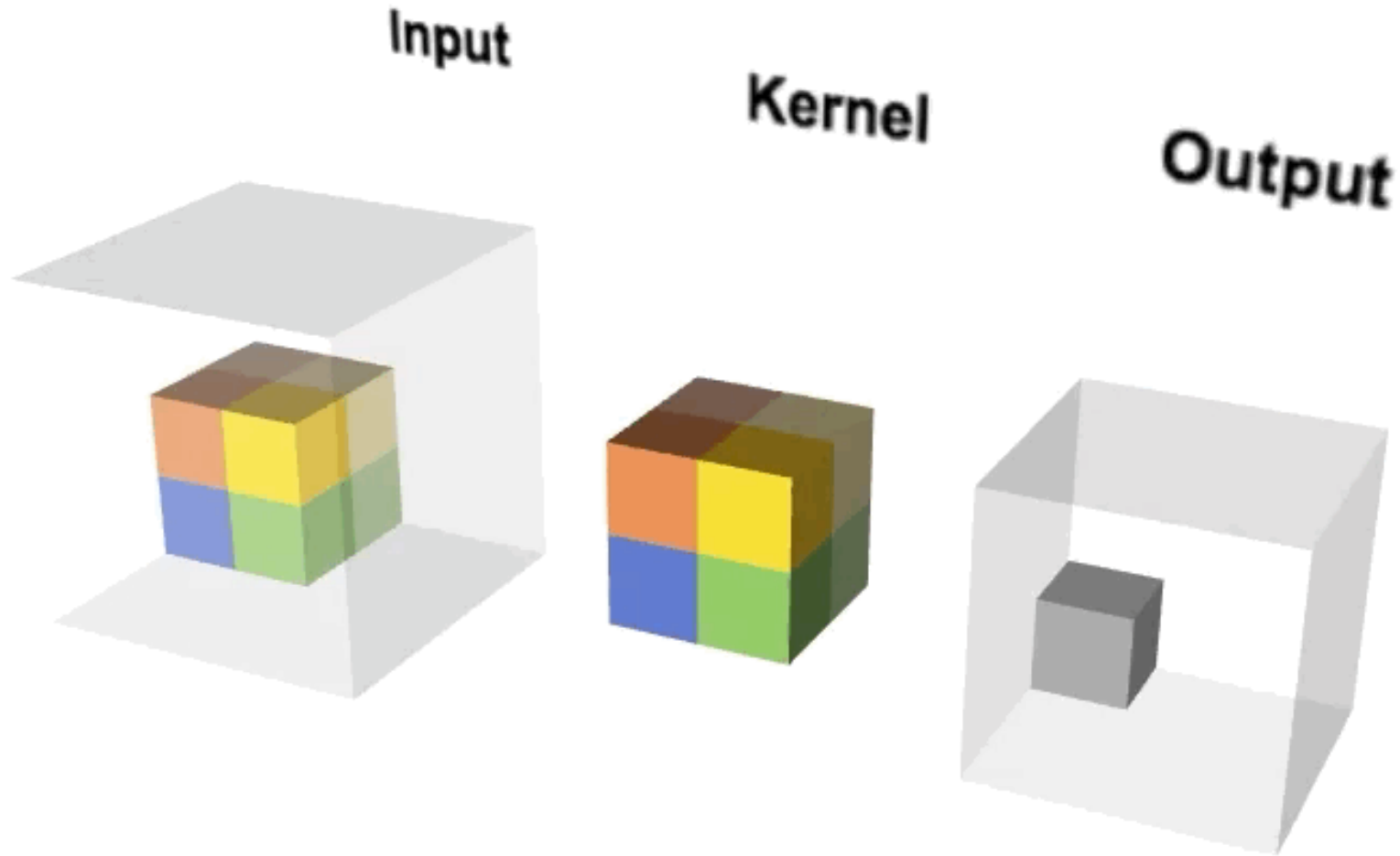


Source: <https://medium.com/data-science/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

3D Convolution



3D Convolution Animation

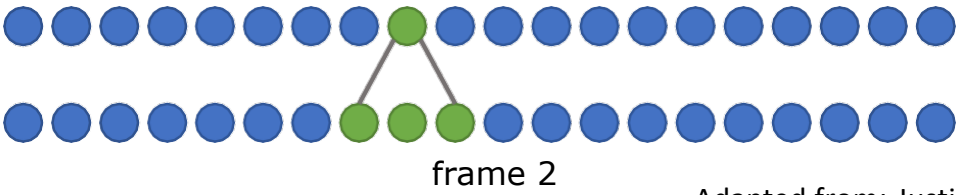
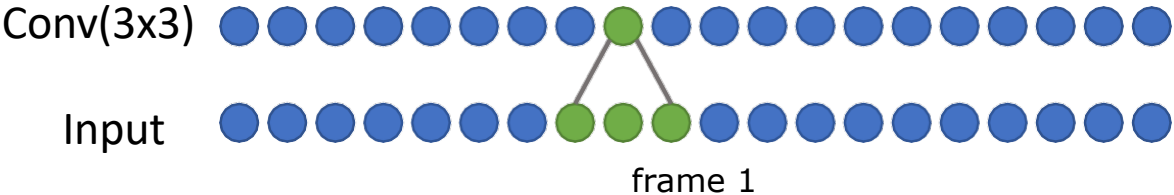


Source: <https://medium.com/apache-mxnet/1d-3d-convolutions-explained-with-ms-excel-5f88c0f35941>

Early Fusion vs Late Fusion vs 3D CNN

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3

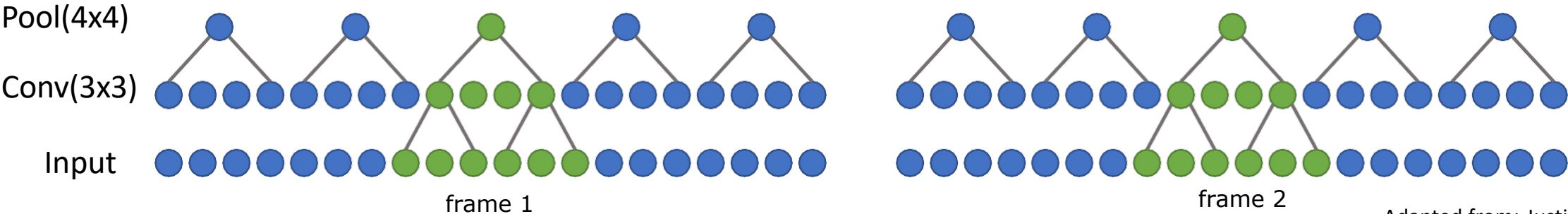
Late
Fusion



Early Fusion vs Late Fusion vs 3D CNN

Late
Fusion

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6

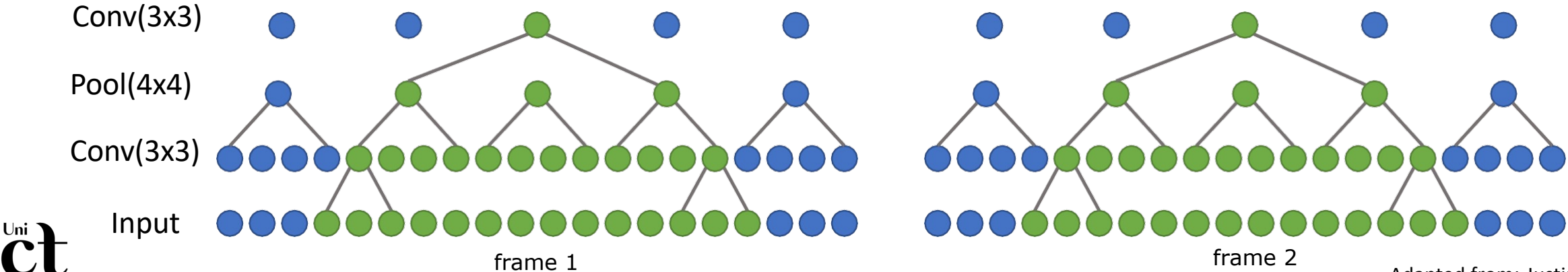


Early Fusion vs Late Fusion vs 3D CNN

Late Fusion

Build slowly in space

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14

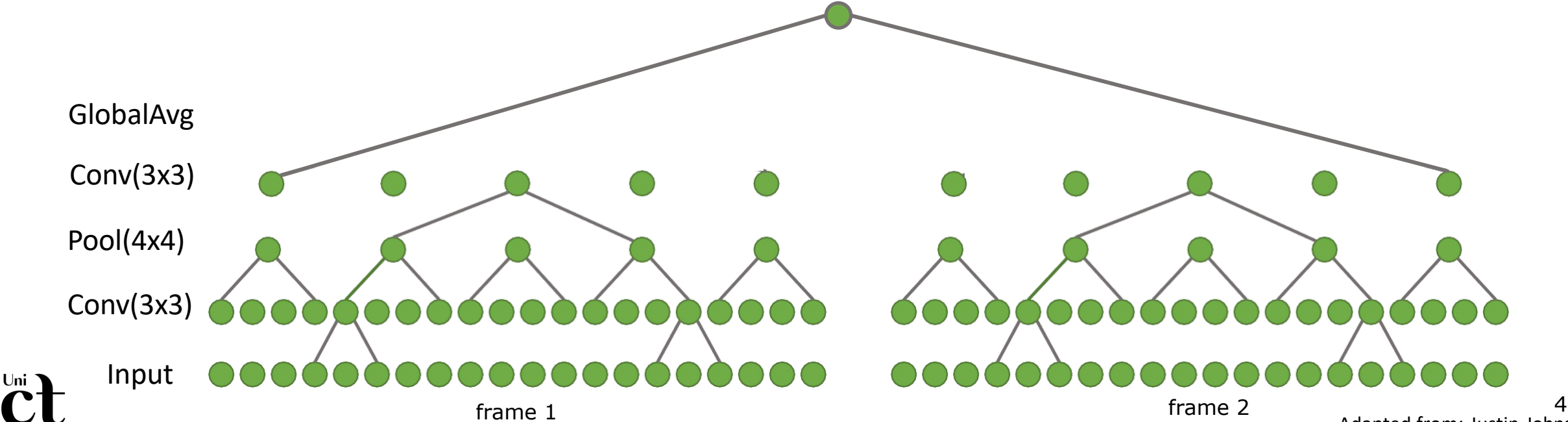


Early Fusion vs Late Fusion vs 3D CNN

Late Fusion

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14
GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64

Build slowly in space,
All-at-once in time at end



Early Fusion vs Late Fusion vs 3D CNN

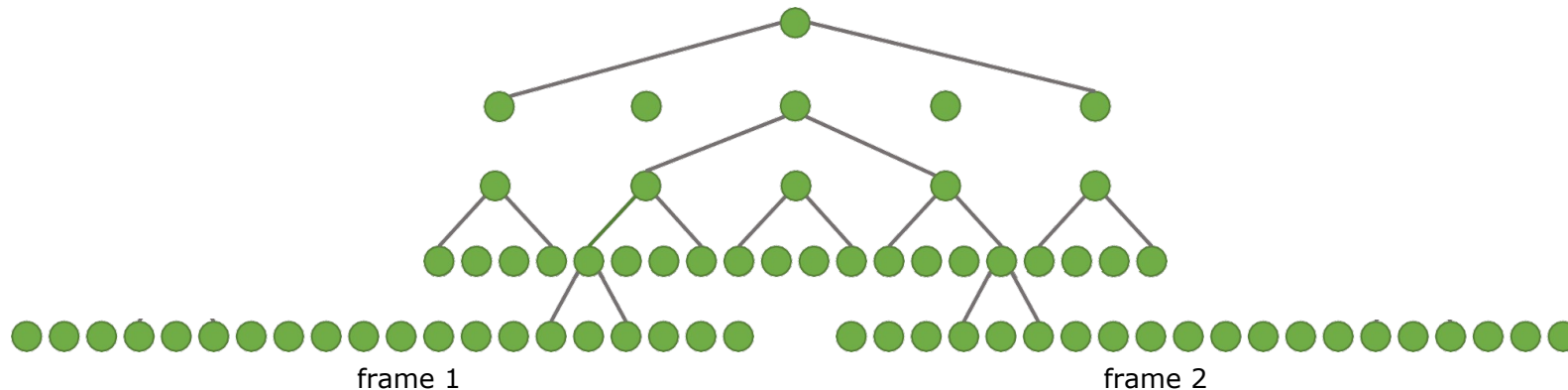
Late Fusion

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14
GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3
Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6
Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14
GlobalAvgPool	24 x 1 x 1	20 x 64 x 64

Build slowly in space,
All-at-once in time at end

Early Fusion

Build slowly in space,
All-at-once in time at start



Early Fusion vs Late Fusion vs 3D CNN

Late Fusion

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14
GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64

Build slowly in space,
All-at-once in time at end

Early Fusion

Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3
Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6
Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14
GlobalAvgPool	24 x 1 x 1	20 x 64 x 64

Build slowly in space,
All-at-once in time at start

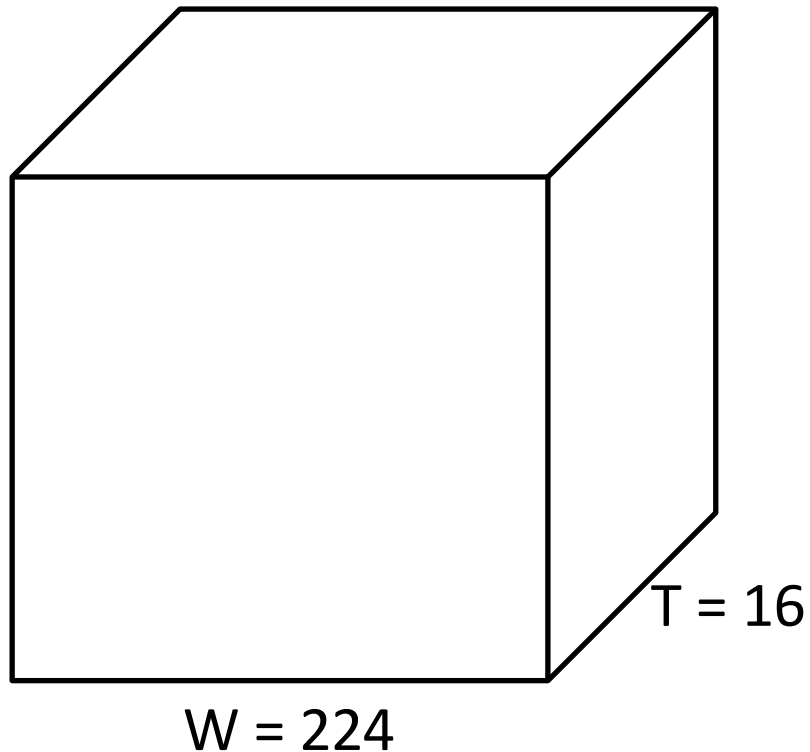
3D CNN

Input	3 x 20 x 64 x 64	
Conv3D(3x3x3, 3->12)	12 x 20 x 64 x 64	3 x 3 x 3
Pool3D(4x4x4)	12 x 5 x 16 x 16	6 x 6 x 6
Conv3D(3x3x3, 12->24)	24 x 5 x 16 x 16	14 x 14 x 14
GlobalAvgPool	24 x 1 x 1	20 x 64 x 64

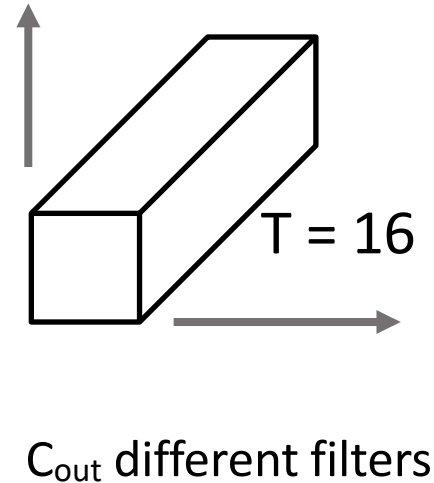
Build slowly in space
Build slowly in time
"Slow Fusion"

2D Conv (Early Fusion) vs 3D Conv (3D CNN)

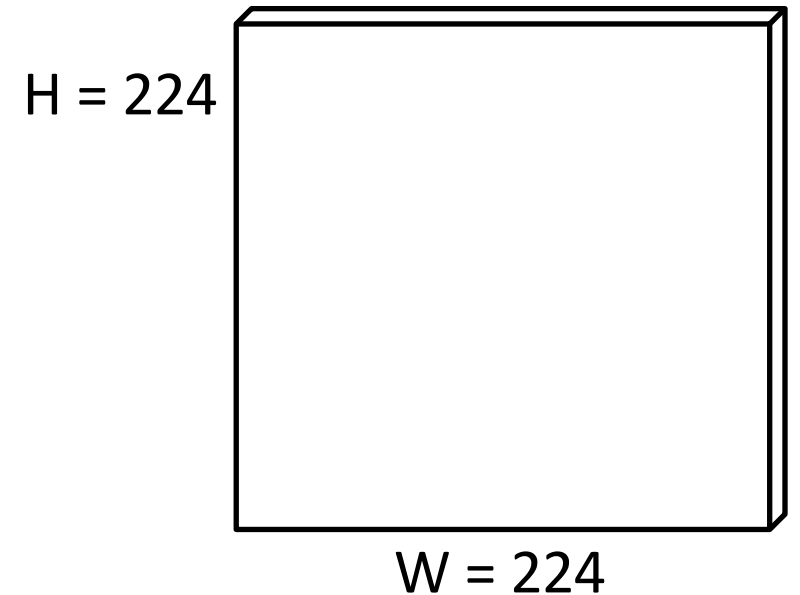
Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



Weight:
 $C_{out} \times C_{in} \times T \times 3 \times 3$
Slide over x and y

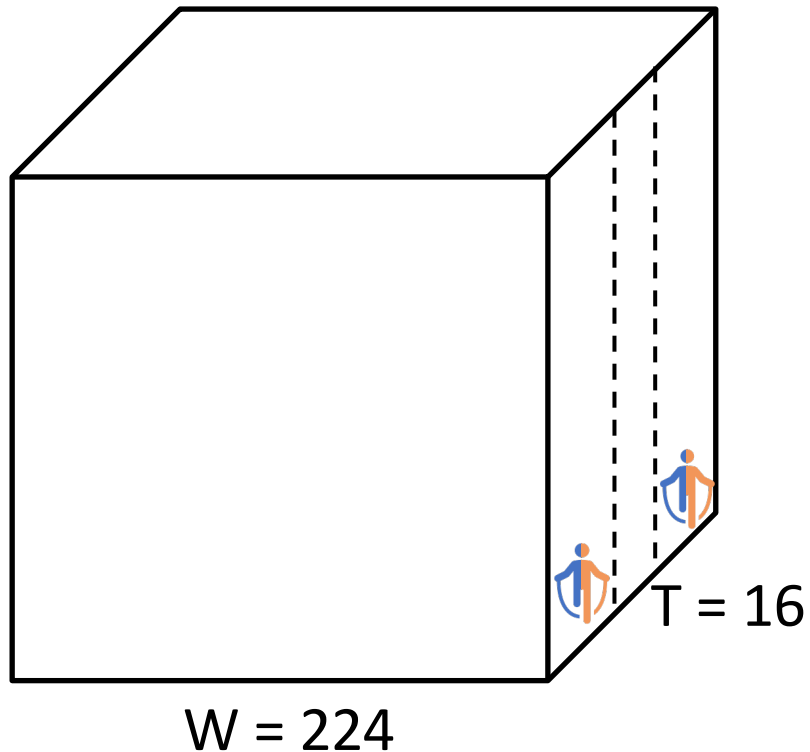


Output:
 $C_{out} \times H \times W$
2D grid with C_{out} -dim
feat at each point

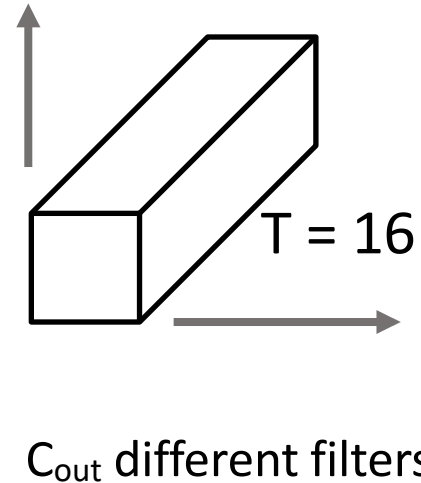


2D Conv (Early Fusion) vs 3D Conv (3D CNN)

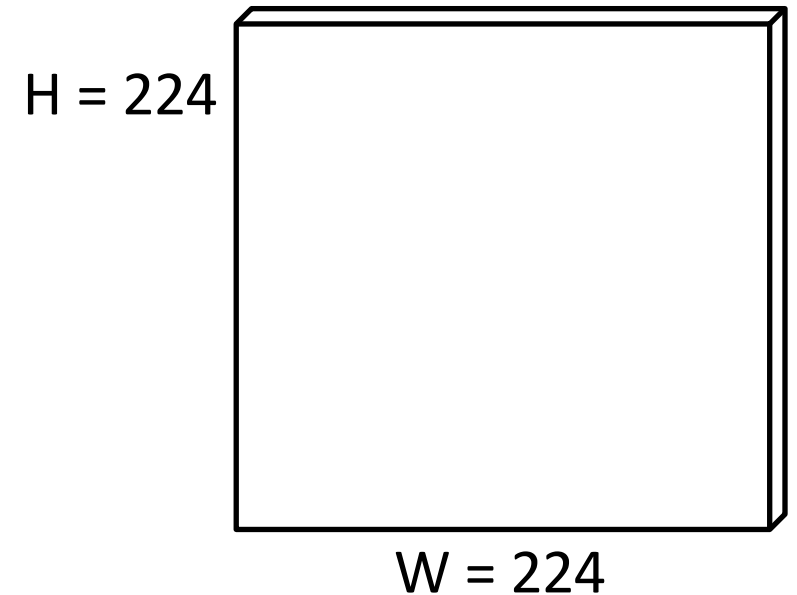
Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



Weight:
 $C_{out} \times C_{in} \times T \times 3 \times 3$
Slide over x and y



Output:
 $C_{out} \times H \times W$
2D grid with C_{out} -dim
feat at each point



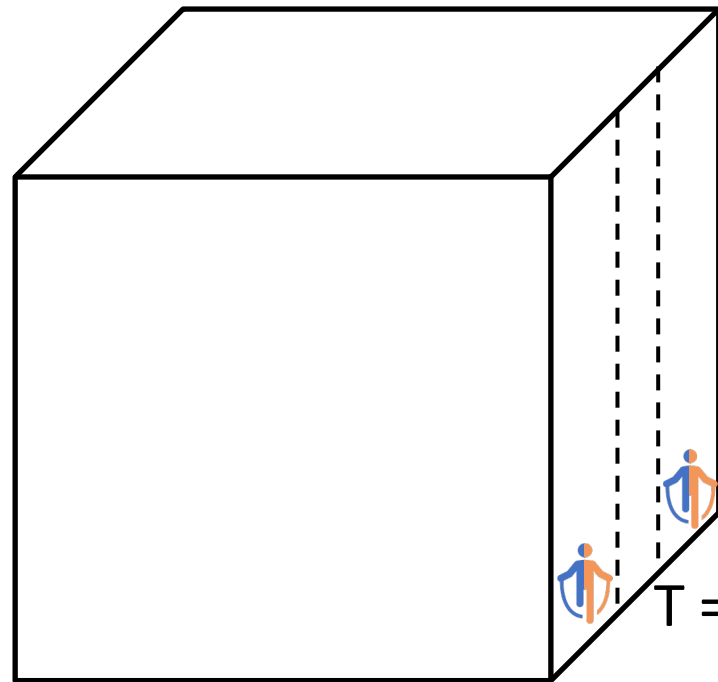
Let's say there are two people jumping, one at the beginning of the clip and another one at the end

2D Conv (Early Fusion) vs 3D Conv (3D CNN)

Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)

Weight:
 $C_{out} \times C_{in} \times T \times 3 \times 3$
Slide over x and y

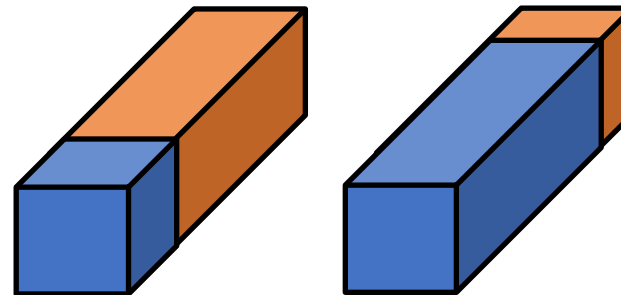
Output:
 $C_{out} \times H \times W$
2D grid with C_{out} -dim



H = 224

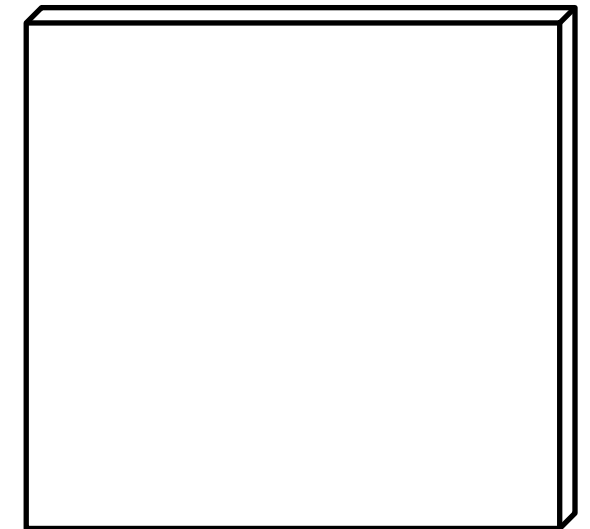
W = 224

T = 16



C_{out} different filters

No temporal shift-invariance! Needs
to learn separate filters for the same
motion at different times in the clip

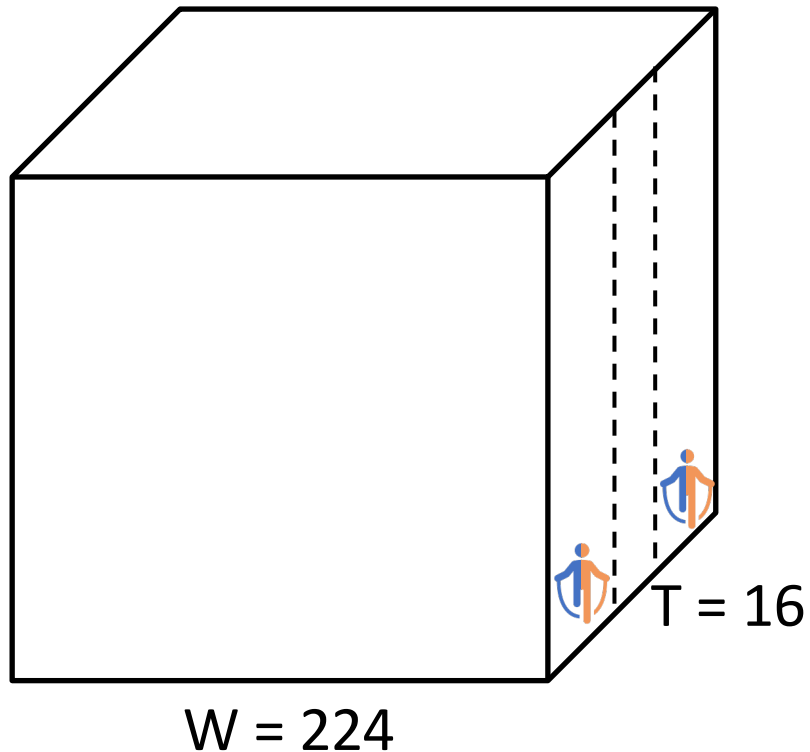


W = 224

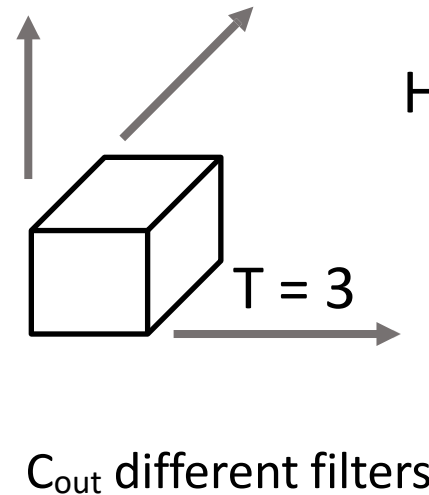
Let's say there are two people jumping, one at the
beginning of the clip and another one at the end

2D Conv (Early Fusion) vs 3D Conv (3D CNN)

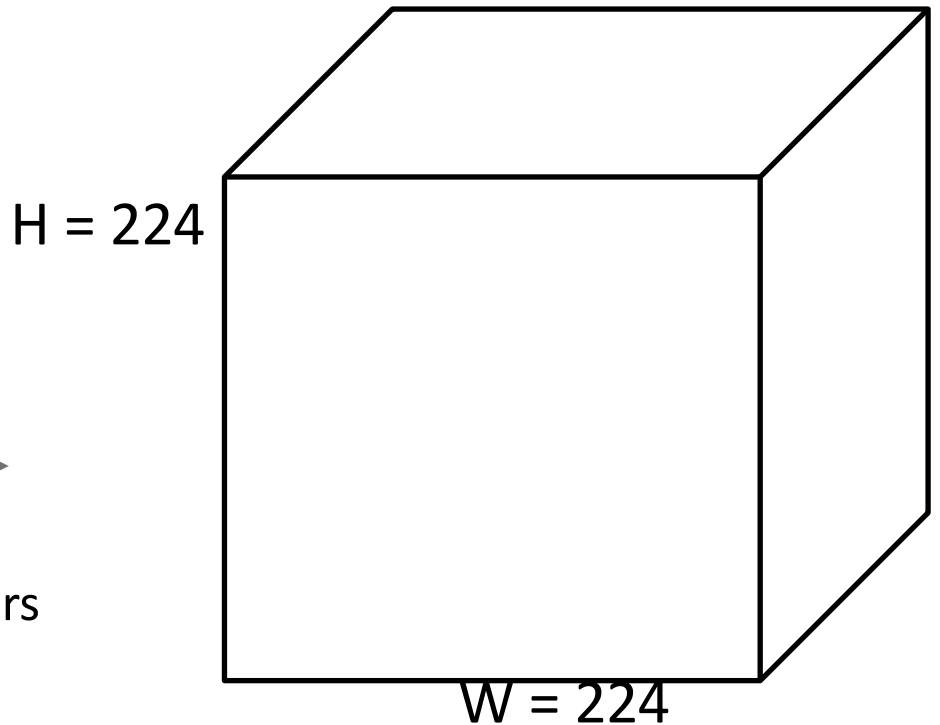
Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



Weight:
 $C_{out} \times C_{in} \times 3 \times 3 \times 3$
Slide over x and y



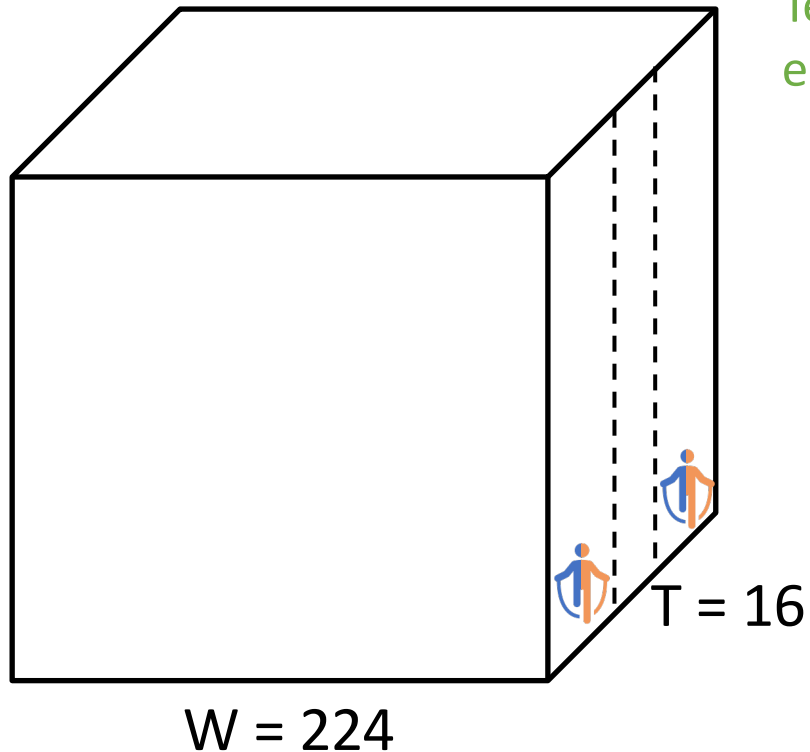
Output:
 $C_{out} \times T \times H \times W$
3D grid with C_{out} -dim
feat at each point



Let's say there are two people jumping, one at the beginning of the clip and another one at the end

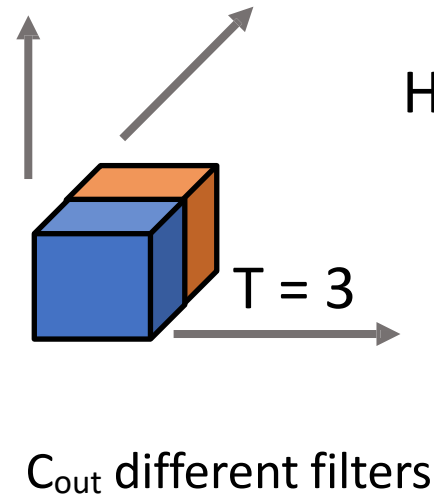
2D Conv (Early Fusion) vs 3D Conv (3D CNN)

Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)

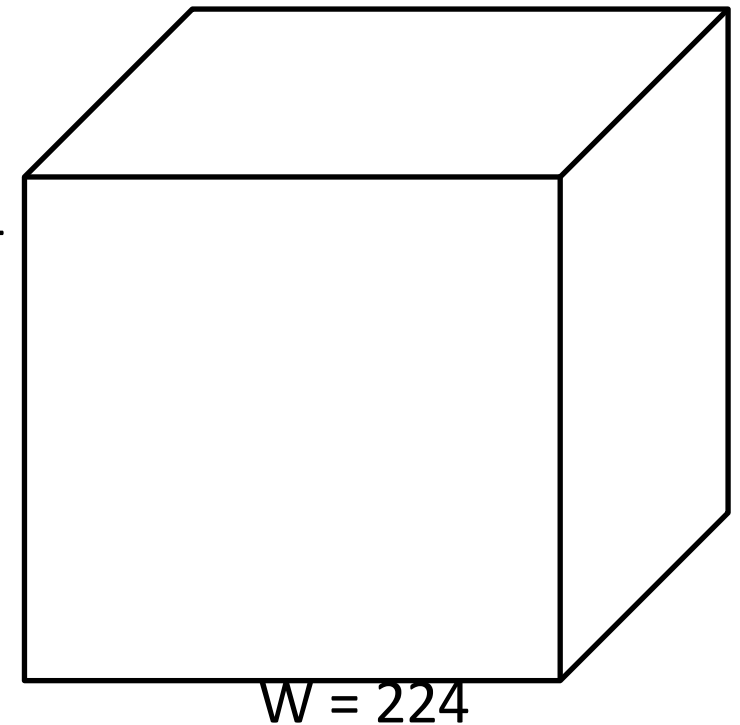


Weight:
 $C_{out} \times C_{in} \times 3 \times 3 \times 3$
Slide over x and y

Temporal shift-invariant since
each filter slides over time!



Output:
 $C_{out} \times T \times H \times W$
3D grid with C_{out} -dim
feat at each point



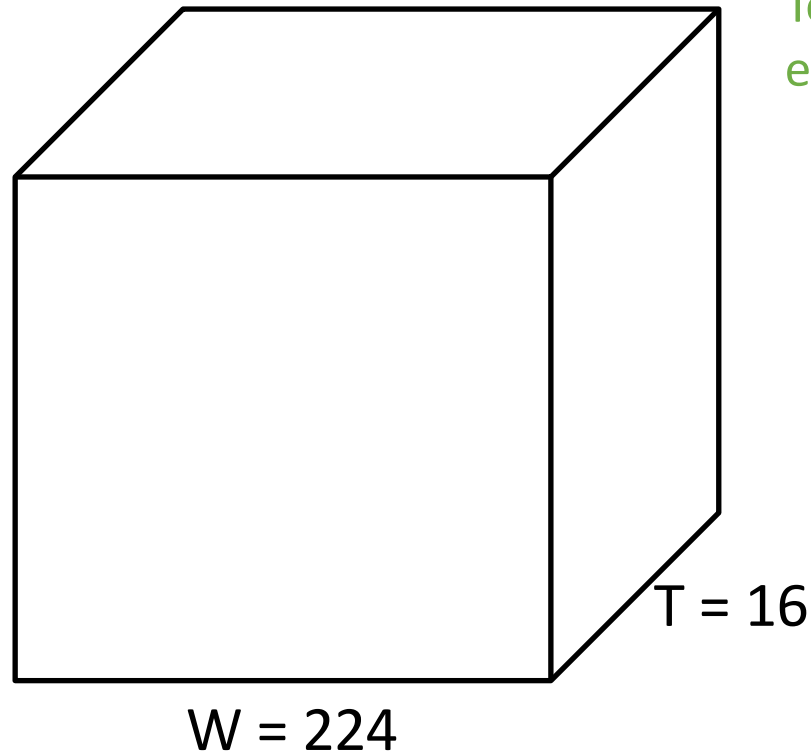
Let's say there are two people jumping, one at the
beginning of the clip and another one at the end

2D Conv (Early Fusion) vs 3D Conv (3D CNN)

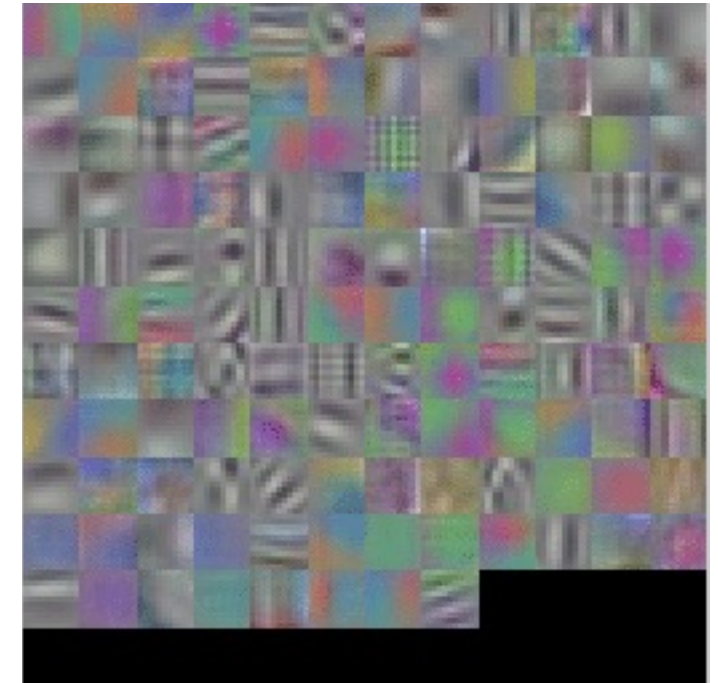
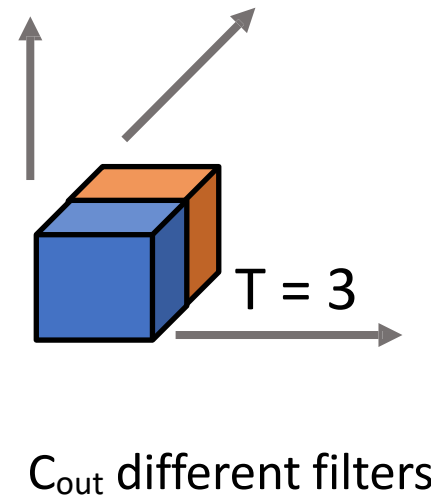
Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)

Weight:
 $C_{out} \times C_{in} \times 3 \times 3 \times 3$
Slide over x and y

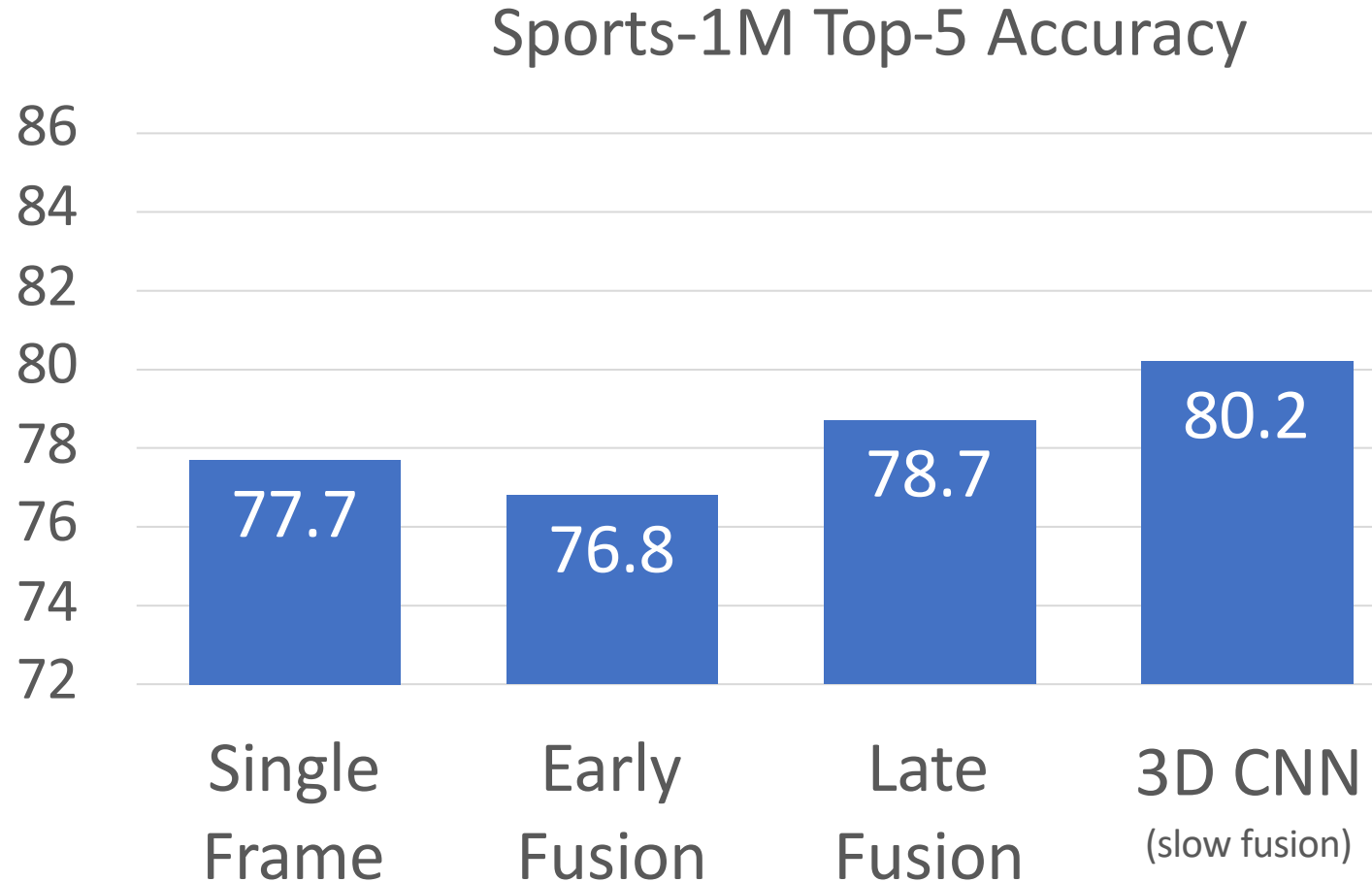
First-layer filters have shape
3 (RGB) x 4 (frames) x 5 x 5 (space)
Can visualize as video clips!



Temporal shift-invariant since
each filter slides over time!



Early Fusion vs Late Fusion vs 3D CNN



Single Frame model works well!

3D CNNs have improved a lot since 2014!

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

C3D: The VGG of 3D CNNs

3D CNN that uses all 3x3x3 conv and
2x2x2 pooling
(except Pool1 which is 1x2x2)

Released model pretrained on Sports-
1M: Many people used this as a video
feature extractor

Layer	Size
Input	3 x 16 x 112 x 112
Conv1 (3x3x3)	64 x 16 x 112 x 112
Pool1 (1x2x2)	64 x 16 x 56 x 56
Conv2 (3x3x3)	128 x 16 x 56 x 56
Pool2 (2x2x2)	128 x 8 x 28 x 28
Conv3a (3x3x3)	256 x 8 x 28 x 28
Conv3b (3x3x3)	256 x 8 x 28 x 28
Pool3 (2x2x2)	256 x 4 x 14 x 14
Conv4a (3x3x3)	512 x 4 x 14 x 14
Conv4b (3x3x3)	512 x 4 x 14 x 14
Pool4 (2x2x2)	512 x 2 x 7 x 7
Conv5a (3x3x3)	512 x 2 x 7 x 7
Conv5b (3x3x3)	512 x 2 x 7 x 7
Pool5	512 x 1 x 3 x 3
FC6	4096
FC7	4096
FC8	C

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

C3D: The VGG of 3D CNNs

3D CNN that uses all 3x3x3 conv and
2x2x2 pooling
(except Pool1 which is 1x2x2)

Released model pretrained on Sports-
1M: Many people used this as a video
feature extractor

Problem: 3x3x3 conv is very expensive!

AlexNet: 0.7 GFLOP

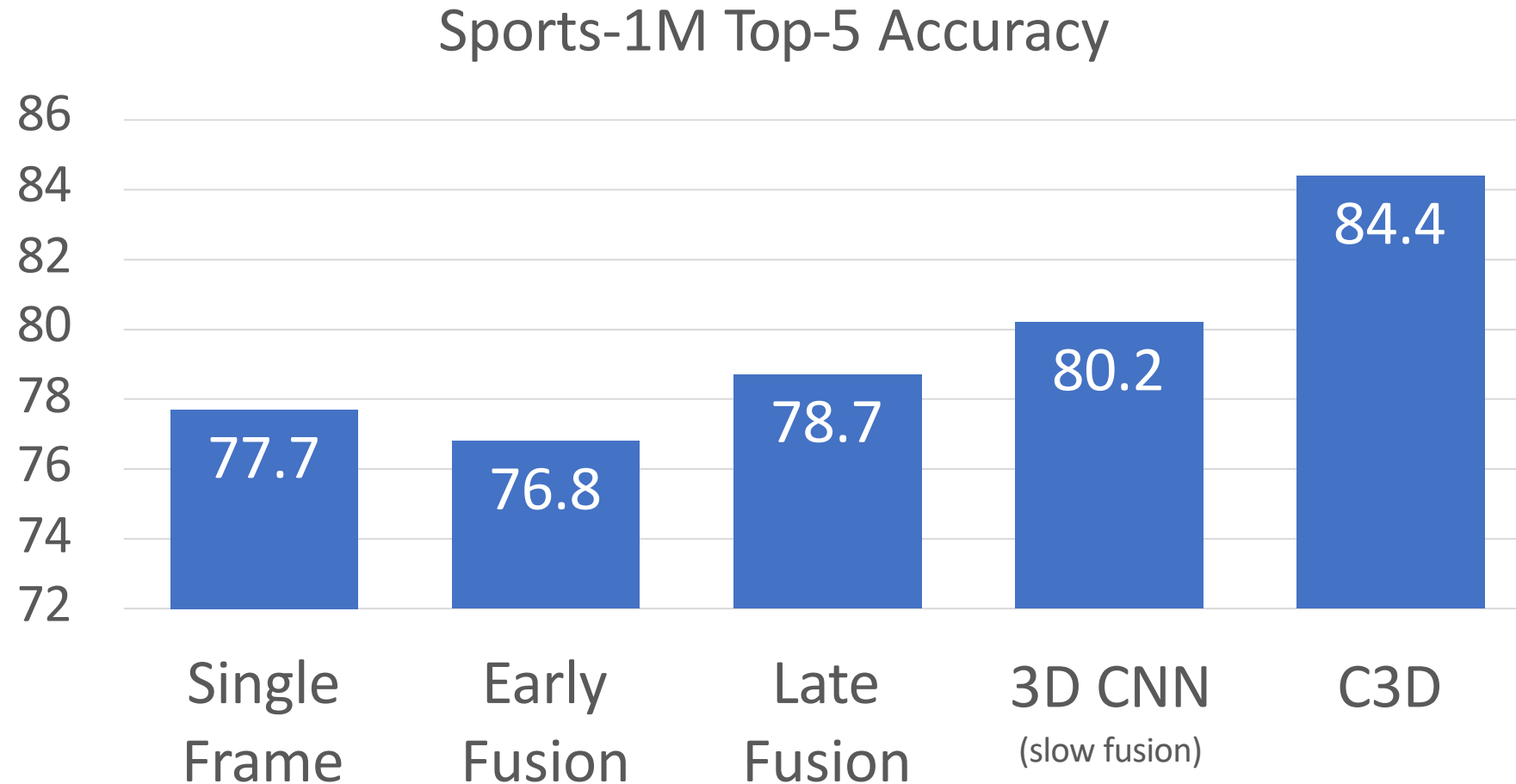
VGG-16: 13.6 GFLOP

C3D: **39.5 GFLOP (2.9x VGG!)**

Layer	Size	MFLOPs
Input	3 x 16 x 112 x 112	
Conv1 (3x3x3)	64 x 16 x 112 x 112	1.04
Pool1 (1x2x2)	64 x 16 x 56 x 56	
Conv2 (3x3x3)	128 x 16 x 56 x 56	11.10
Pool2 (2x2x2)	128 x 8 x 28 x 28	
Conv3a (3x3x3)	256 x 8 x 28 x 28	5.55
Conv3b (3x3x3)	256 x 8 x 28 x 28	11.10
Pool3 (2x2x2)	256 x 4 x 14 x 14	
Conv4a (3x3x3)	512 x 4 x 14 x 14	2.77
Conv4b (3x3x3)	512 x 4 x 14 x 14	5.55
Pool4 (2x2x2)	512 x 2 x 7 x 7	
Conv5a (3x3x3)	512 x 2 x 7 x 7	0.69
Conv5b (3x3x3)	512 x 2 x 7 x 7	0.69
Pool5	512 x 1 x 3 x 3	
FC6	4096	0.51
FC7	4096	0.45
FC8	C	0.05

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

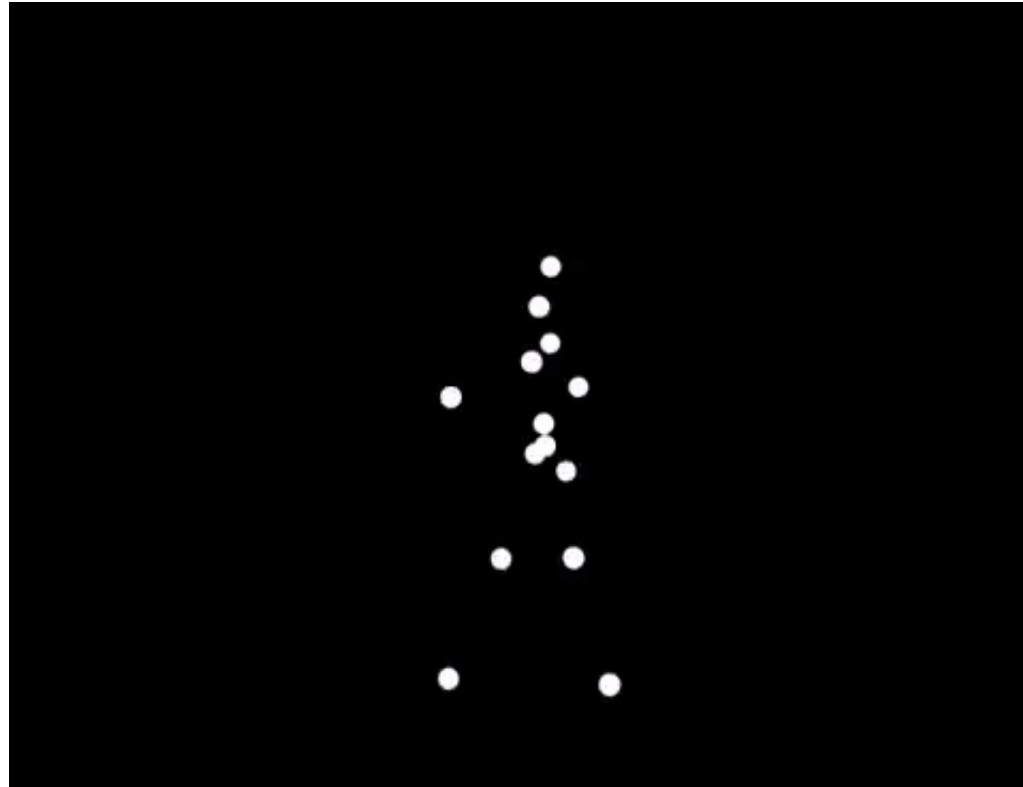
Early Fusion vs Late Fusion vs 3D CNN



Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014
Tran et al, "Learning Spatiotemporal Features with 3D Convolutional Networks", ICCV 2015

Back to Basics: Modeling Motion Explicitly

We can easily recognize actions using only **motion information**



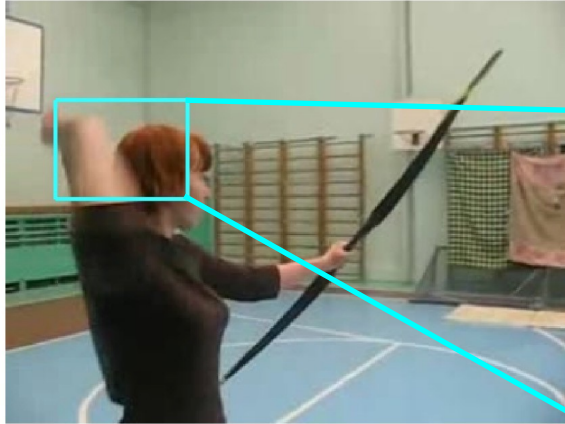
Johansson, "Visual perception of biological motion and a model for its analysis." *Perception & Psychophysics*. 14(2):201-211. 1973.

Measuring Motion with Optical Flow

Optical flow highlights local motion

Can be encoded as a set of two images

Image at frame t



Optical flow gives a displacement field F between images I_t and I_{t+1}

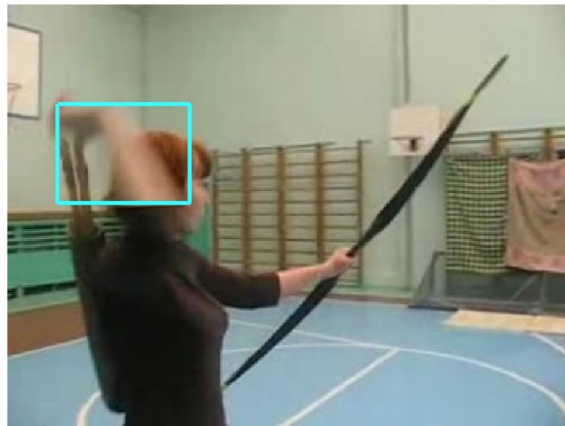
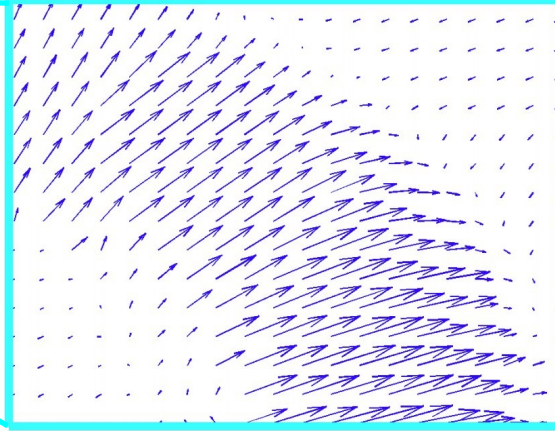


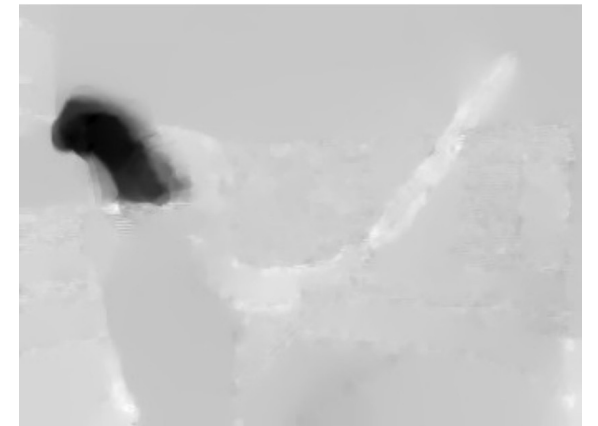
Image at frame $t+1$

Tells where each pixel will move in the next frame:

$$F(x, y) = (dx, dy)$$

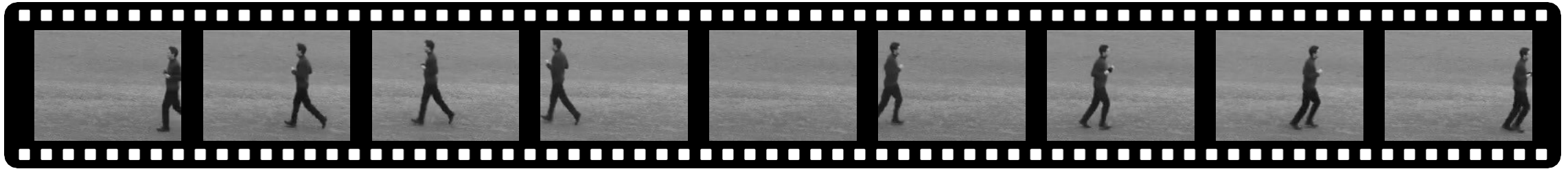
$$I_{t+1}(x+dx, y+dy) = I_t(x, y)$$

Horizontal flow dx



Vertical Flow dy

Different ways to see video



1) Single Image



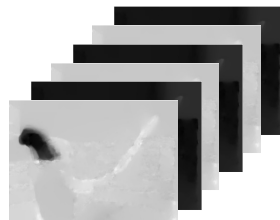
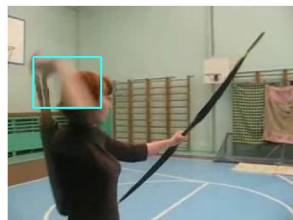
2) A Bag of Images



3) 3D (x,y,t) volume

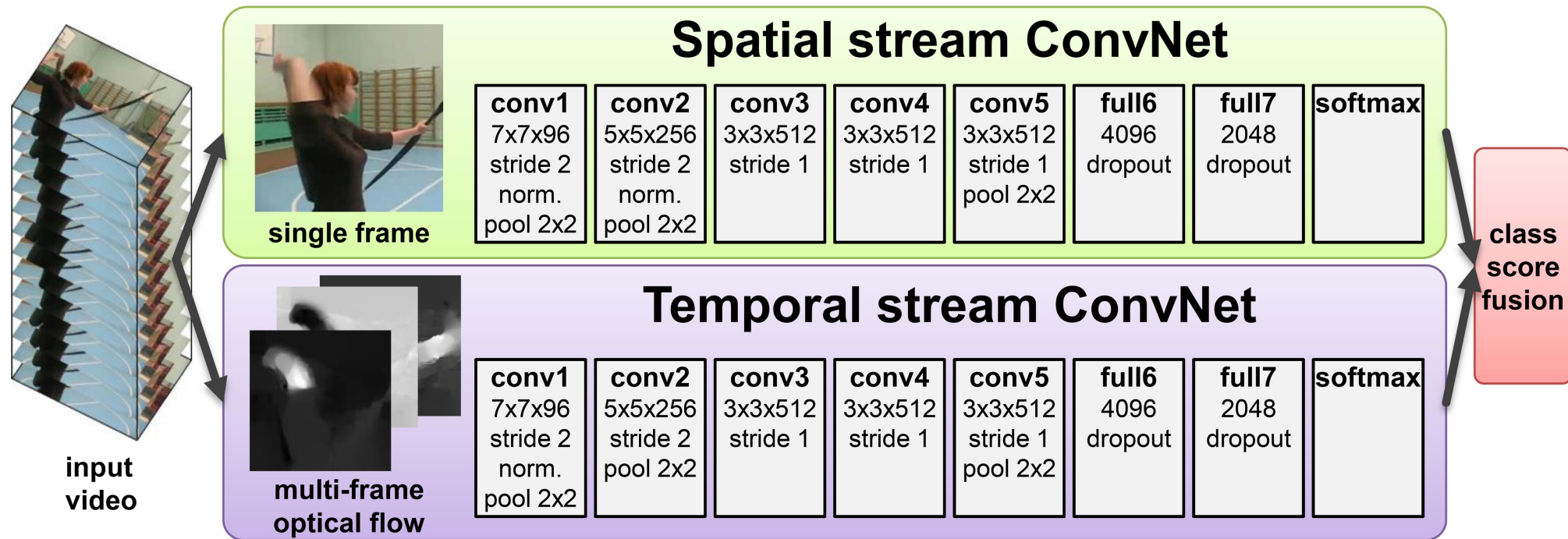


4) Single Image + Multi-Frame Optical Flow



Separating Motion and Appearance: Two-Stream Networks

Input: Single Image
 $3 \times H \times W$

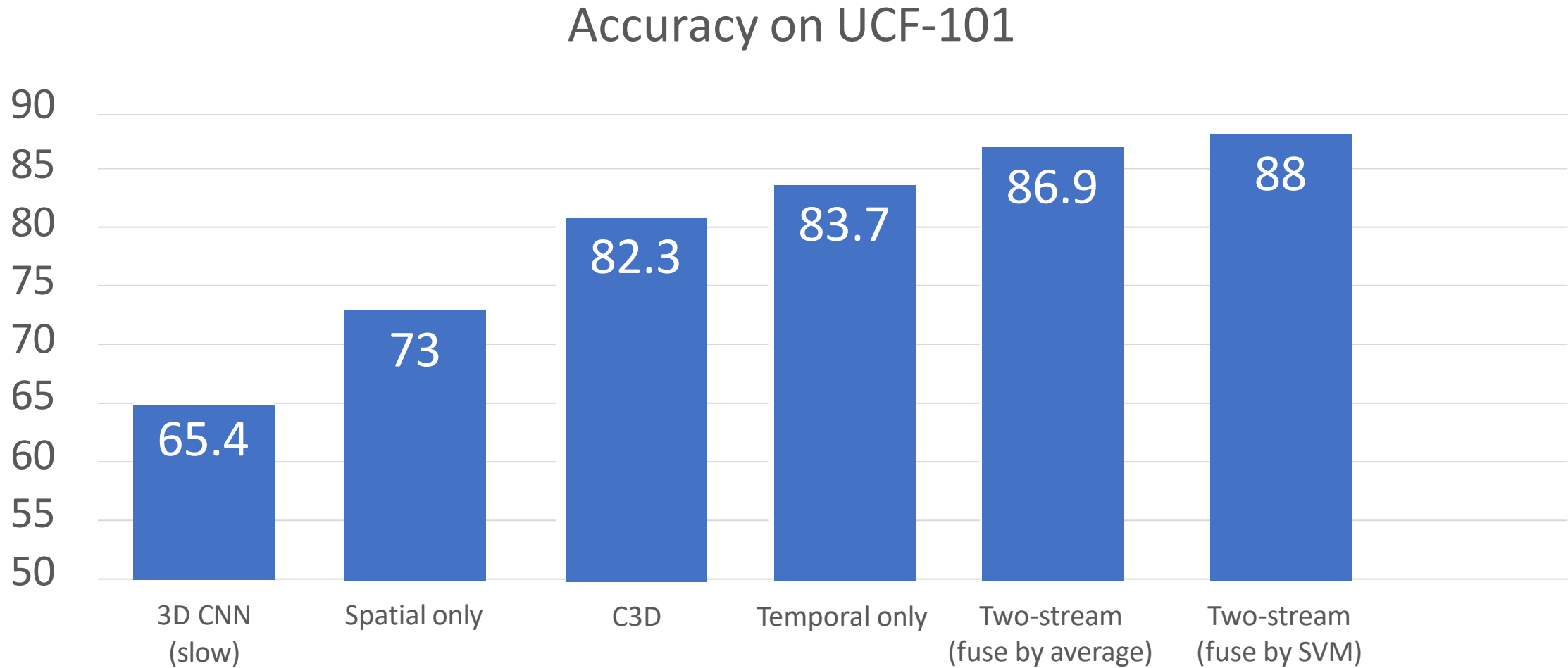


Input: Stack of optical flow:
 $[2*(T-1)] \times H \times W$

Early fusion: First 2D conv
processes all flow images

Simonyan and Zisserman, "Two-stream convolutional networks for action recognition in videos", NeurIPS 2014

Separating Motion and Appearance: Two-Stream Networks



Different ways to see video



1) Single Image



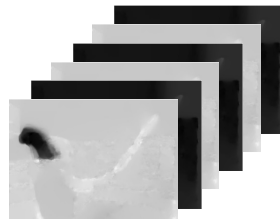
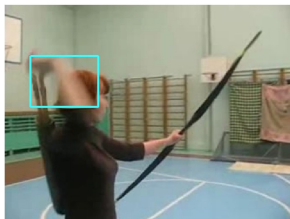
2) A Bag of Images



3) 3D (x,y,t) volume



4) Single Image + Multi-Frame Optical Flow

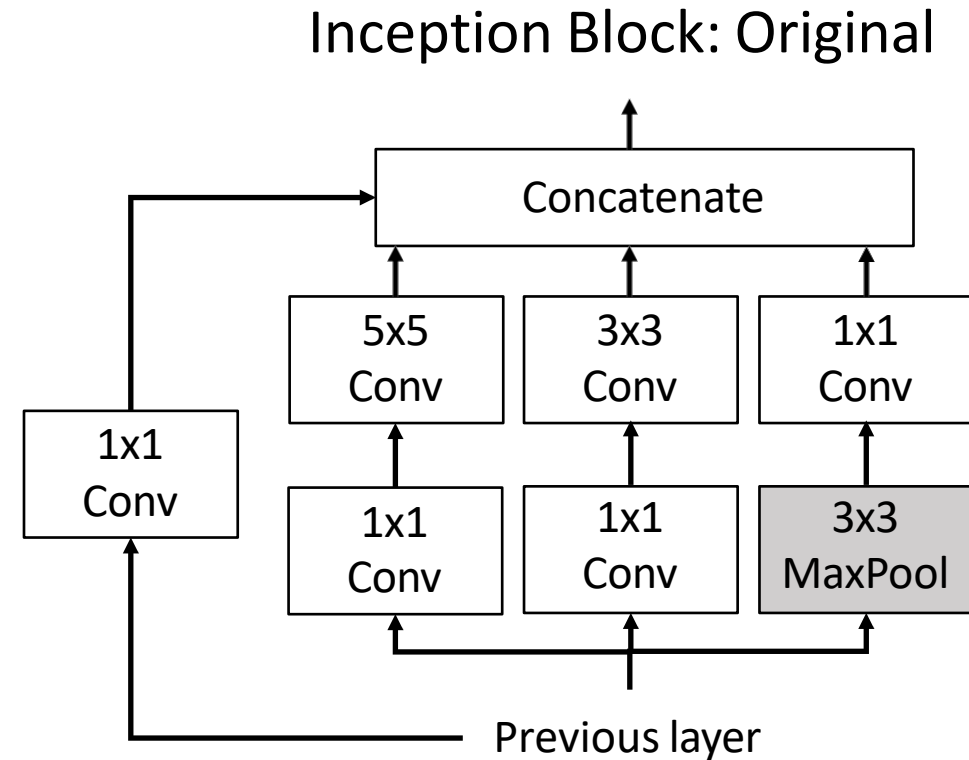


Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.
Can we reuse image architectures for video?

Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool layer with a 3D $K_t \times K_h \times K_w$ version



Carreira and Zisserman, "Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset", CVPR 2017

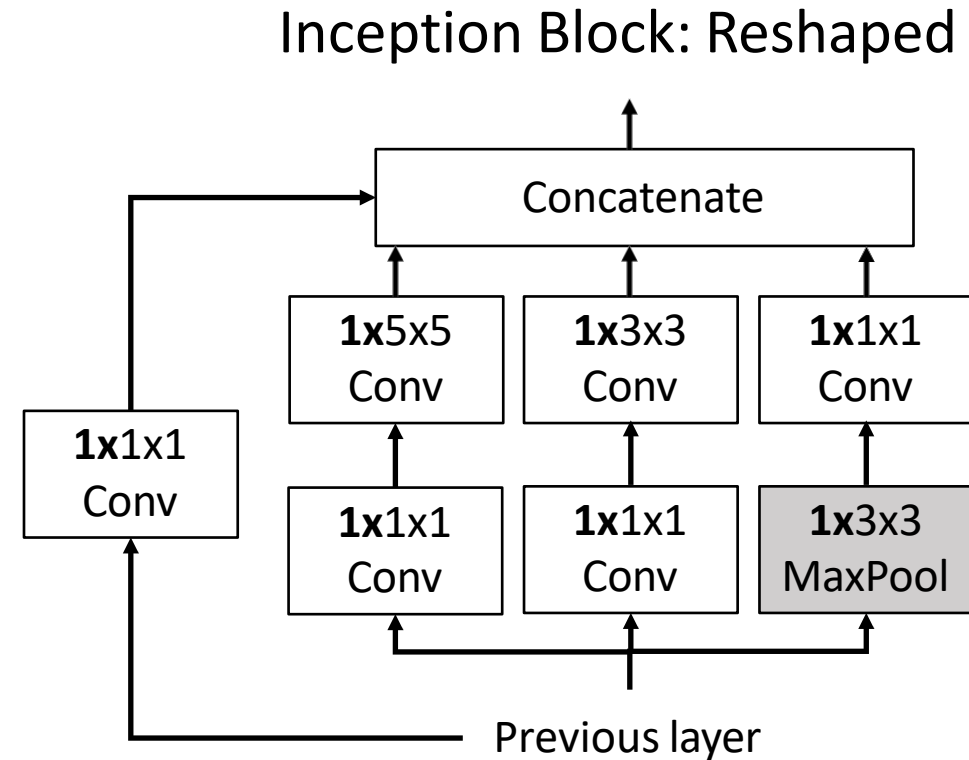
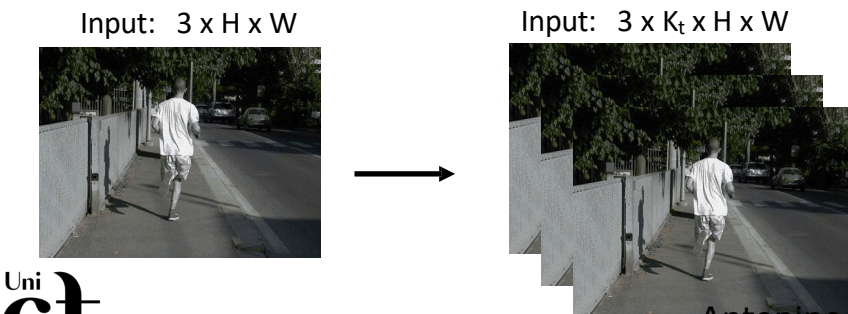
Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.
Can we reuse image architectures for video?

Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool layer with a 3D $K_t \times K_h \times K_w$ version

Step 1: assume images are 1-frame videos and «reshape» all kernels



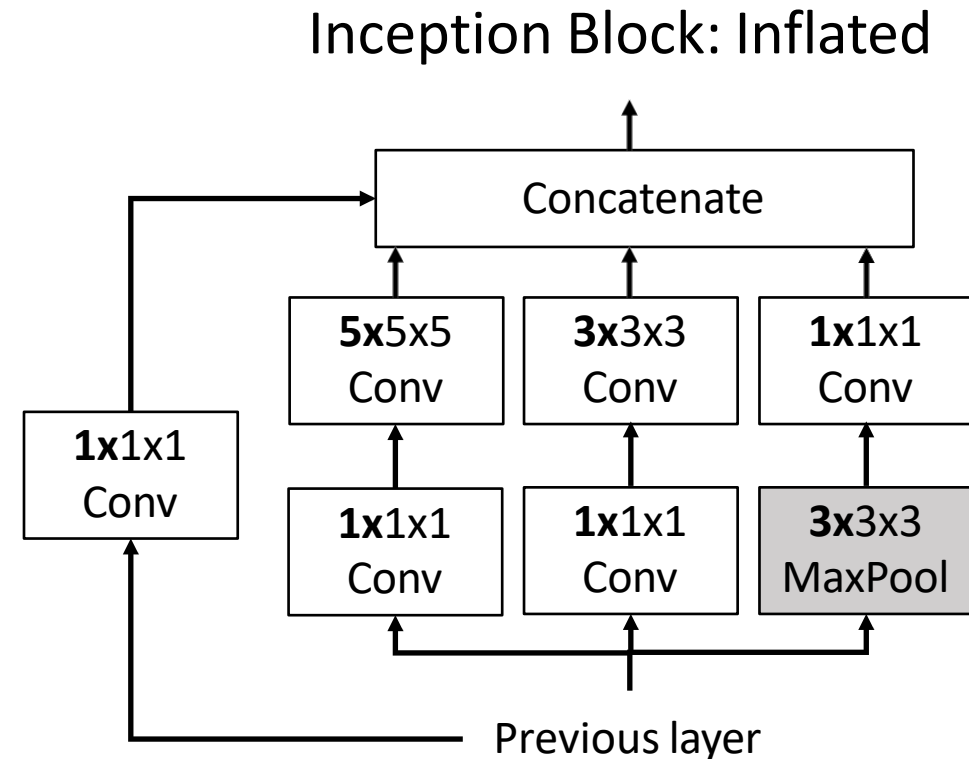
Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.
Can we reuse image architectures for video?

Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool layer with a 3D $K_t \times K_h \times K_w$ version

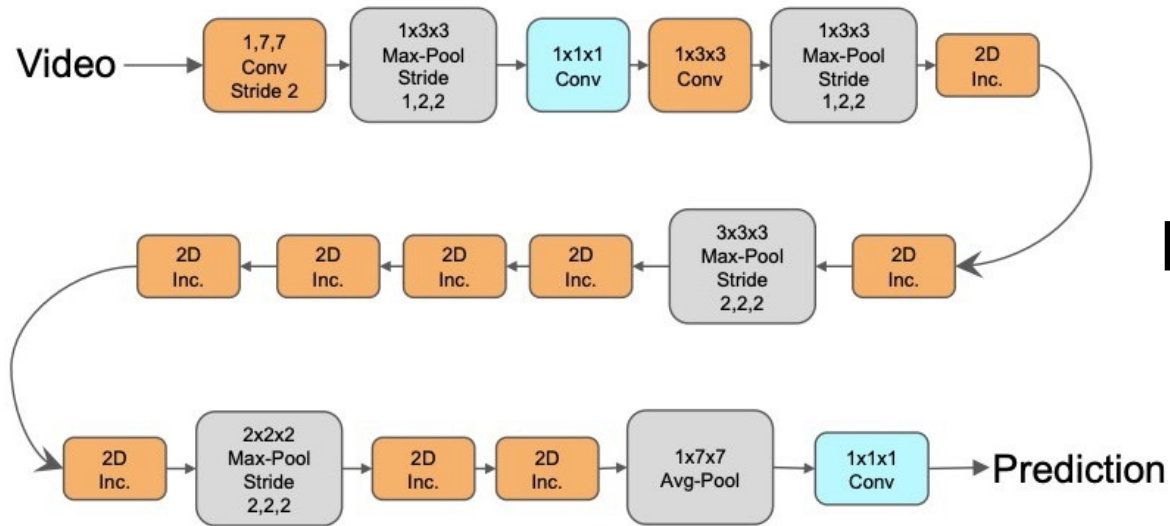
Step 2: assume images are N-frame videos where all frames are identical – «inflate» the filters



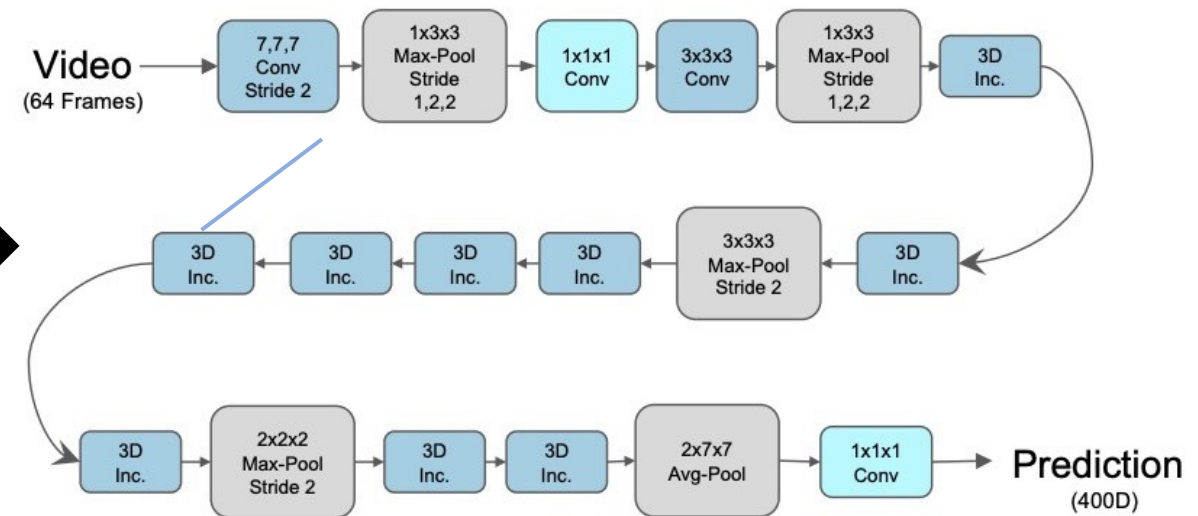
Carreira and Zisserman, "Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset", CVPR 2017

Inflating 2D Networks to 3D (I3D)

2D InceptionV1 (I2D)



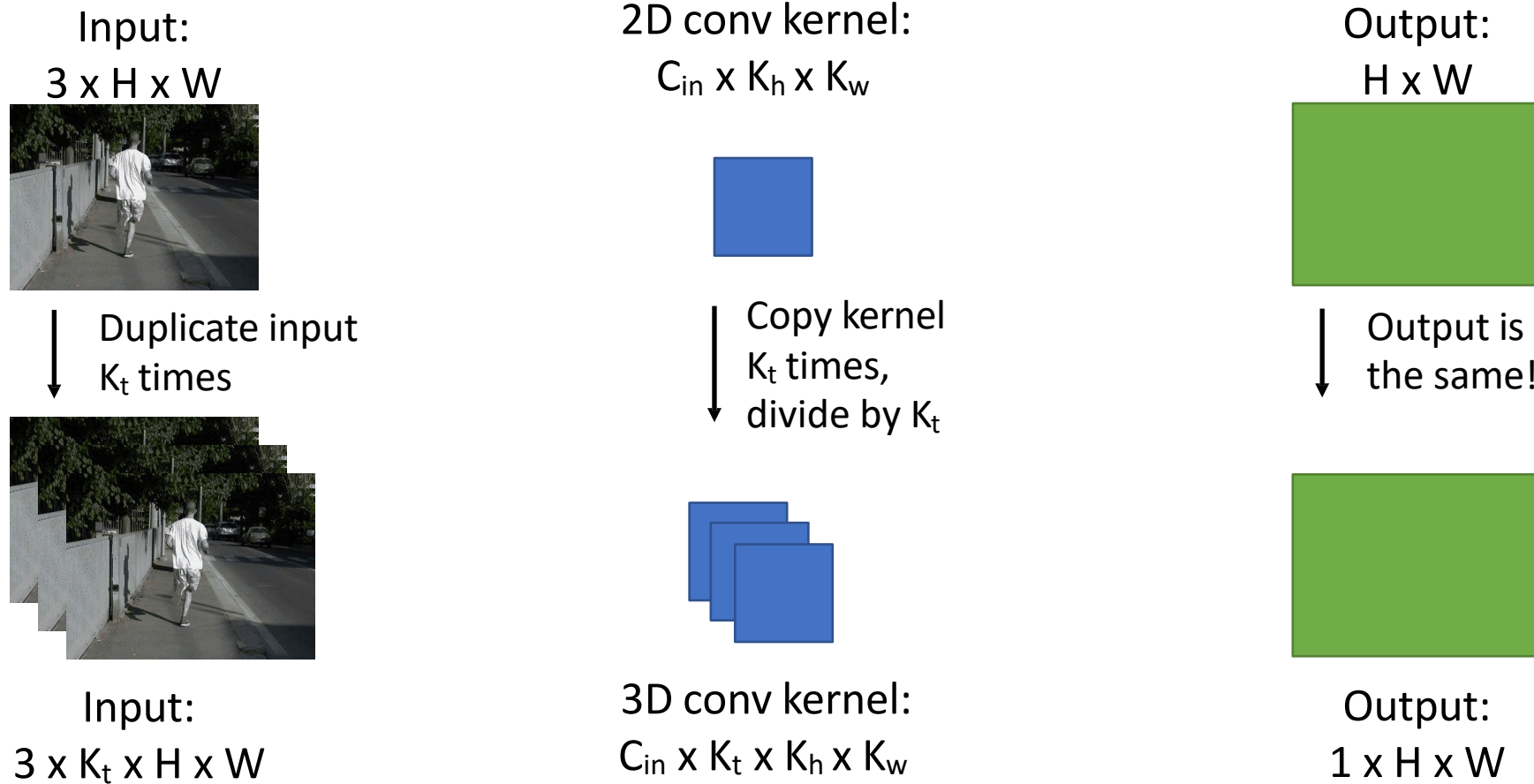
3D InceptionV1 (I3D)



Carreira and Zisserman, "Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset", CVPR 2017

Inflating 2D Networks to 3D (I3D)

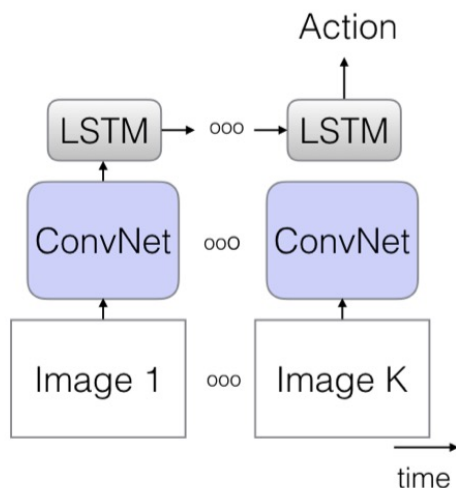
If we assume that all frames are the same, we can initialize I3D with the weights of InceptionV1 just replicating along the temporal dimension and dividing by the number of frames



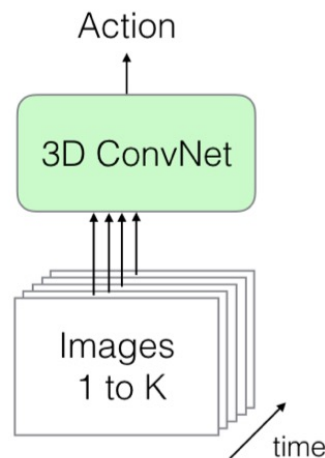
This crucially enables pre-training from large-scale imagenet

Quo Vadis, Action Recognition?

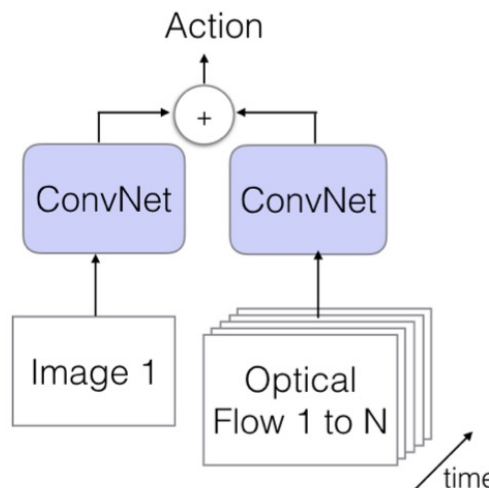
a) LSTM



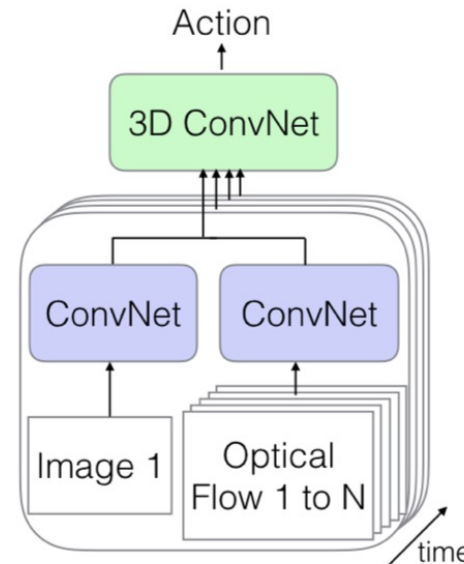
b) 3D-ConvNet



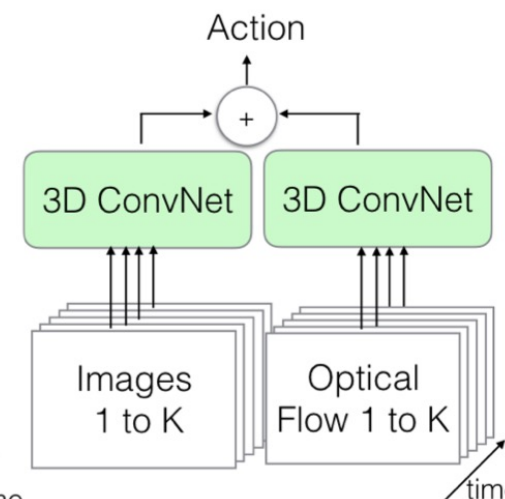
c) Two-Stream



d) 3D-Fused Two-Stream



e) Two-Stream 3D-ConvNet



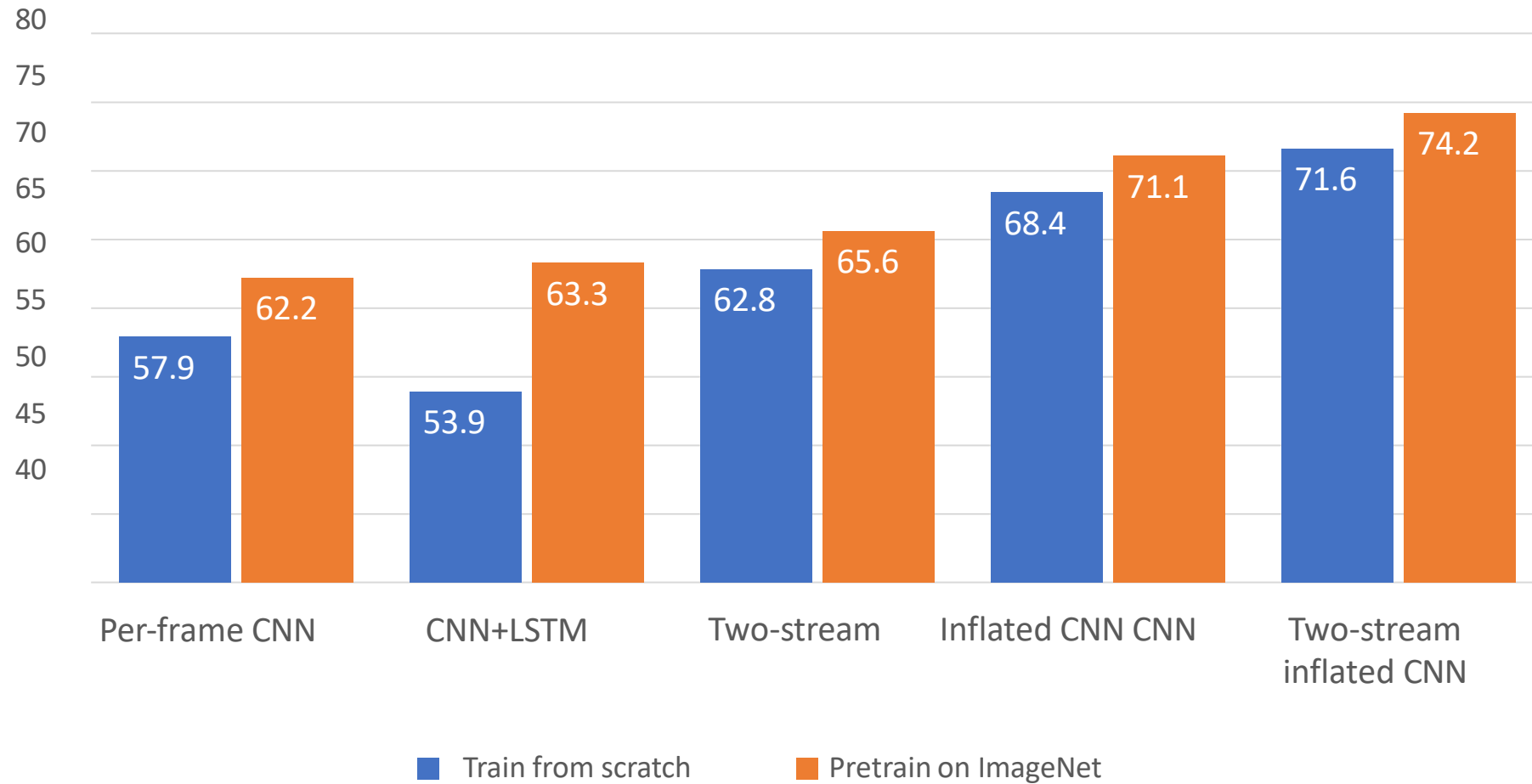
more on this in the next lecture

The paper also benchmarks 5 approaches to action recognition

Carreira and Zisserman, "Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset", CVPR 2017

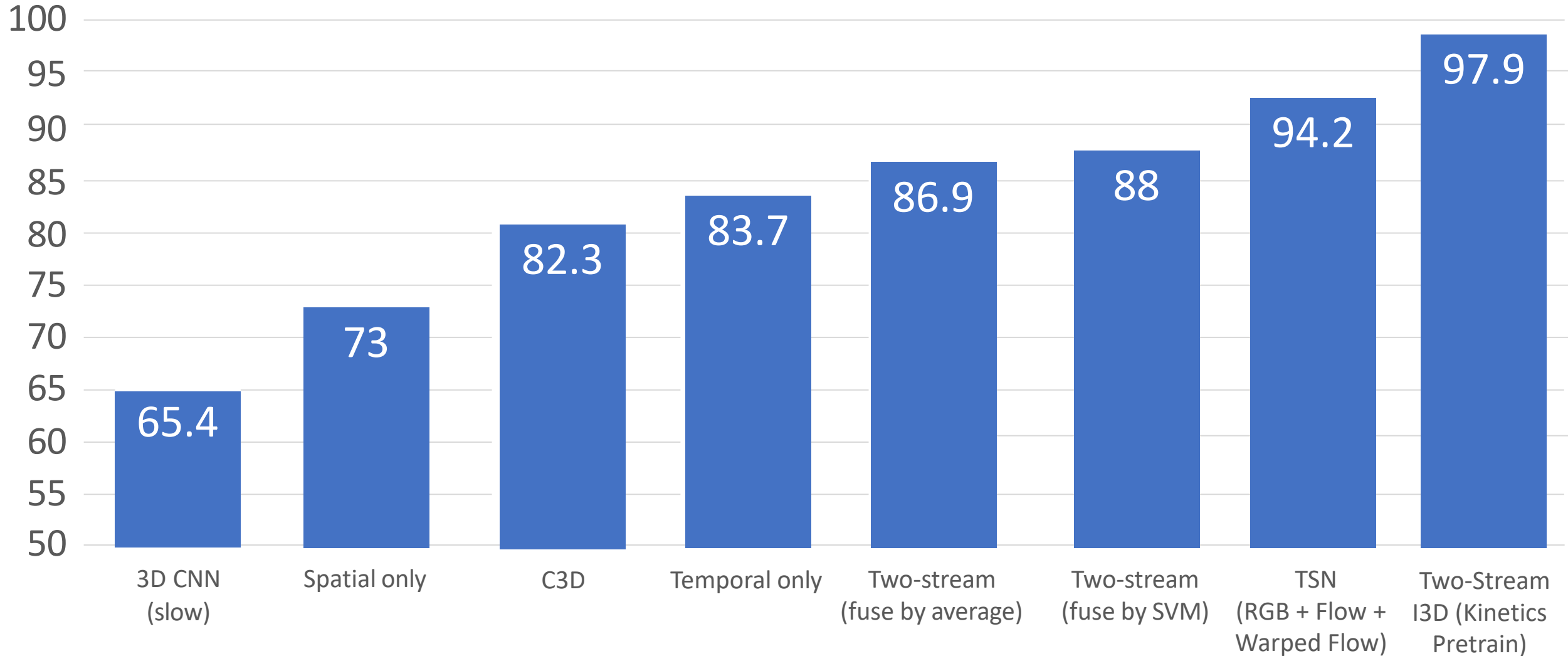
Inflating 2D Networks to 3D (I3D)

Top-1 Accuracy on Kinetics-400



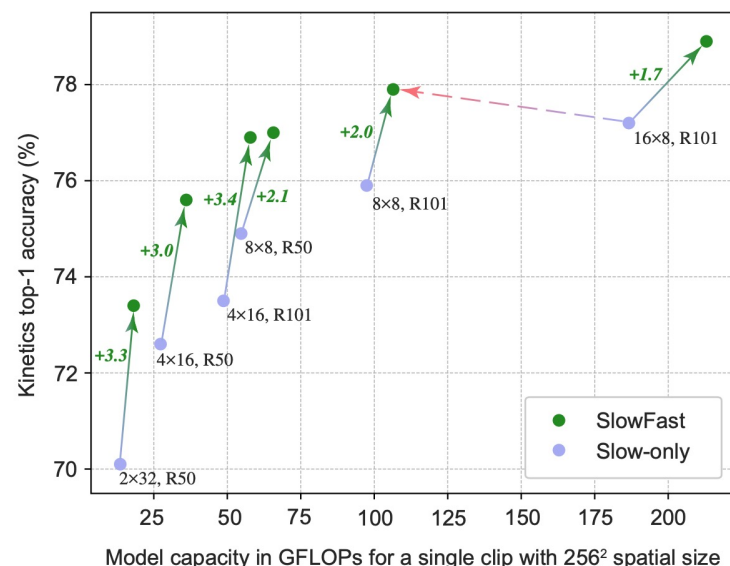
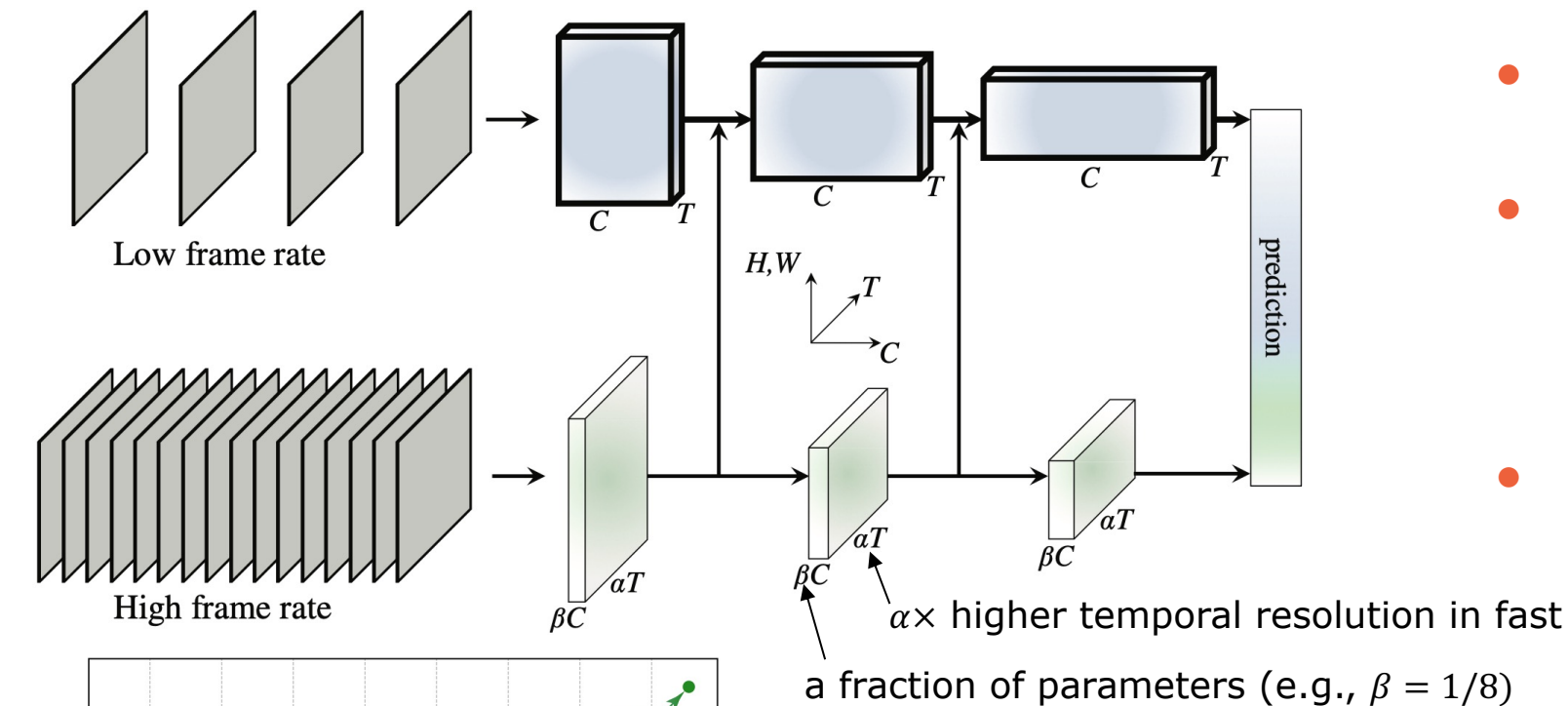
Inflating 2D Networks to 3D (I3D)

Accuracy on UCF-101



SlowFast Networks

C. Feichtenhofer, H. Fan, J. Malik, e K. He, «SlowFast Networks for Video Recognition», 29 2019, *arXiv*: arXiv:1812.03982.

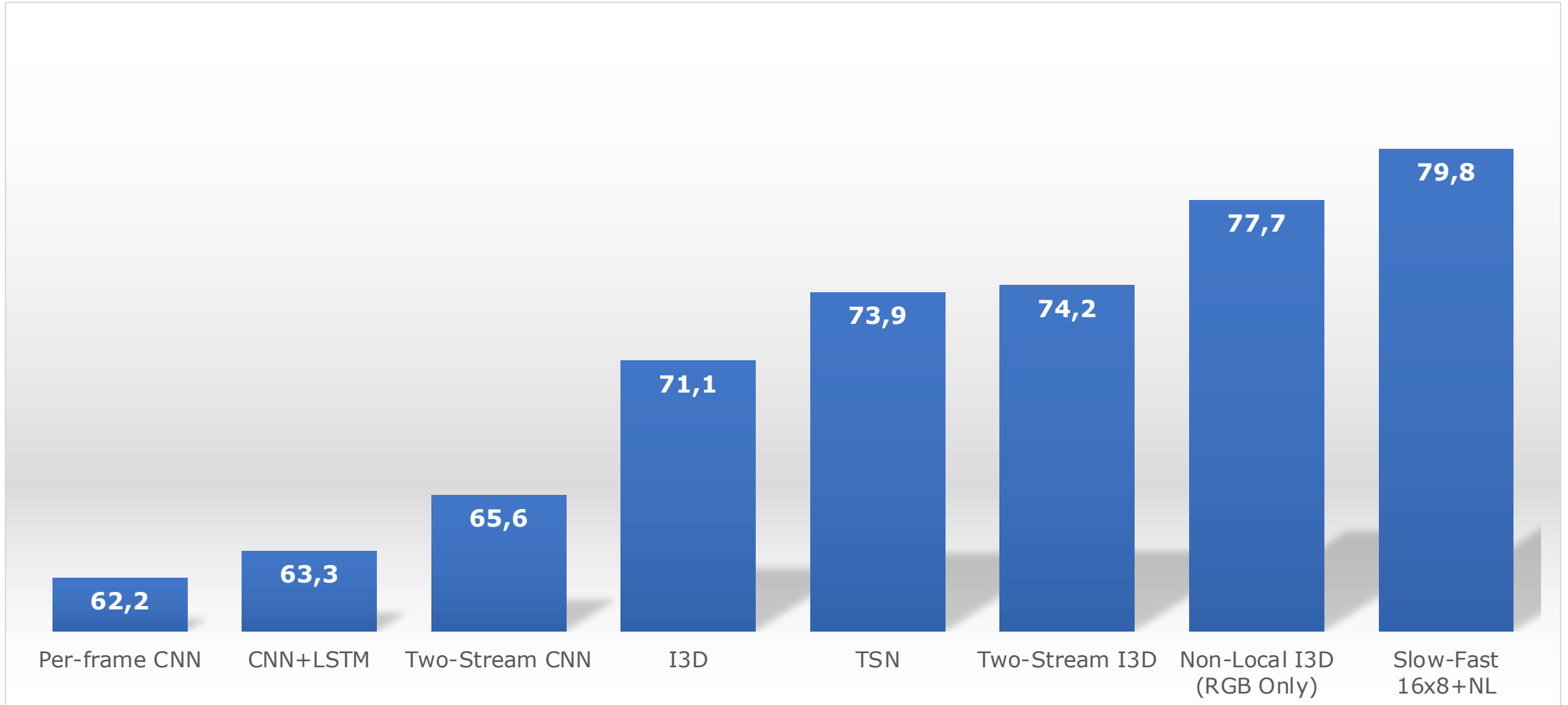


The «fast» branch is light due to the smaller number of parameters but brings performance improvements compared to slow only

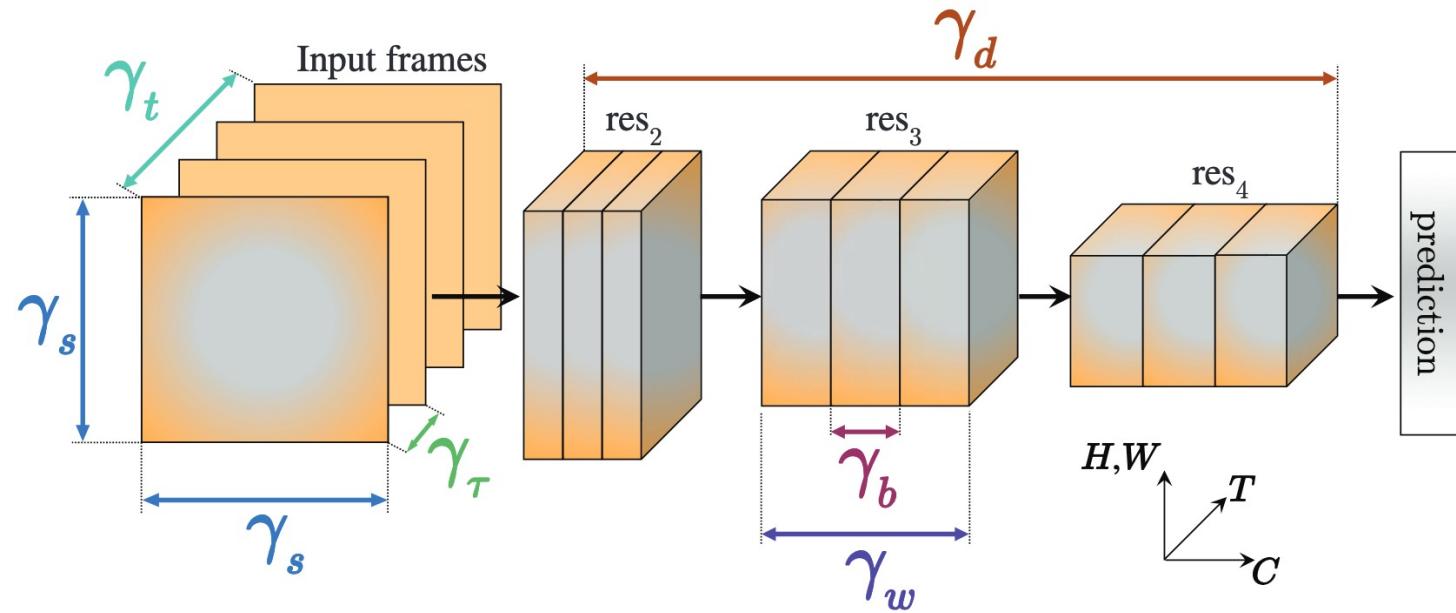
- 3D CNNs are not that efficient at modeling motion
- Indeed, we need to feed them with optical flow, while, in principle, they should be able to extract motion information on their own
- Idea – two streams:
 - «Slow» stream with low frame-rate but more channels focuses on appearance
 - «Fast» stream with high framerate, less channels, which focuses on motion
- Lateral connections allow communications
- Based on ResNet-50 architectures with nonlocal blocks

SlowFast Networks

Top-1 Accuracy on Kinetics-400



X3D: Expanding Architectures for Efficient Video Recognition



model	top-1	top-5	regime FLOPs (G)	FLOPs (G)	Params (M)
X3D-XS	68.6	87.9	$X\text{-Small} \leq 0.6$	0.60	3.76
X3D-S	72.9	90.5	$Small \leq 2$	1.96	3.76
X3D-M	74.6	91.7	$Medium \leq 5$	4.73	3.76
X3D-L	76.8	92.5	$Large \leq 20$	18.37	6.08
X3D-XL	78.4	93.6	$X\text{-Large} \leq 40$	35.84	11.0
SlowFast	79.8	93.9		234	59.9

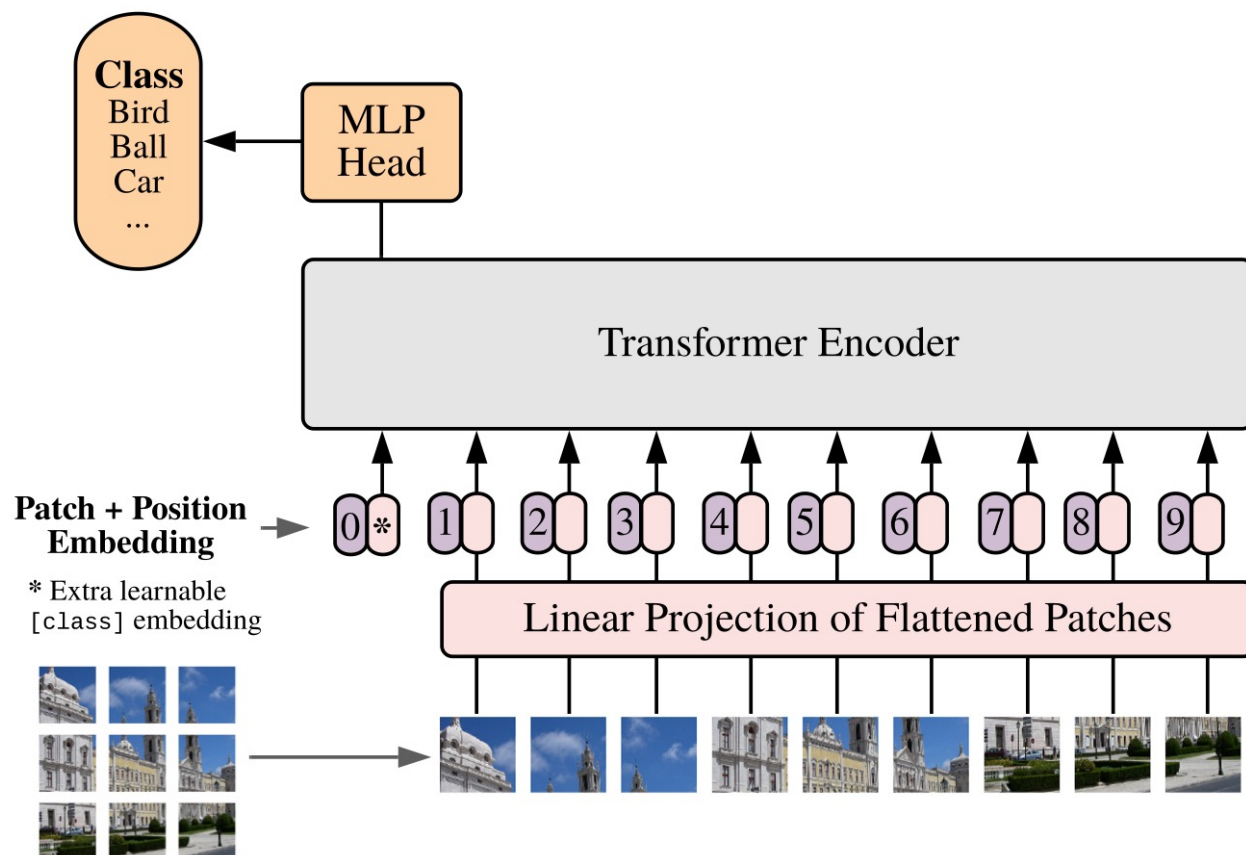
- How to deliver efficient 3D CNNs?
- Instantiate an initial X2D architecture, then let it «expand» in different dimensions:
 - Spatial resolution
 - Temporal resolution
 - Number of frames
 - Stride
 - Number of residual blocks
 - Number of channels
 - Residual block bottleneck width
- Coordinate descent procedure to perform *expansion*, *contraction* and assess goodness of current configuration
- Allows to obtain different configurations subject to budget

What about Transformers?

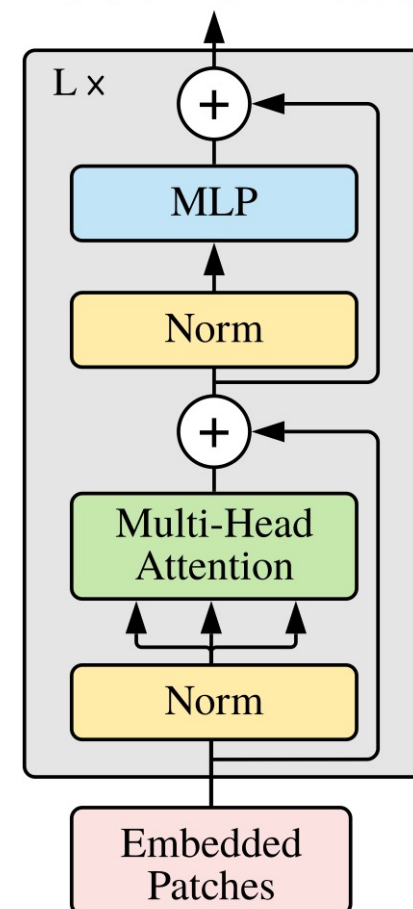
Vision Transformer for 2D inputs (ViT)

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani et al. "An image is worth 16x16 words: Transformers for image recognition at scale." *arXiv preprint arXiv:2010.11929* (2020).

Vision Transformer (ViT)

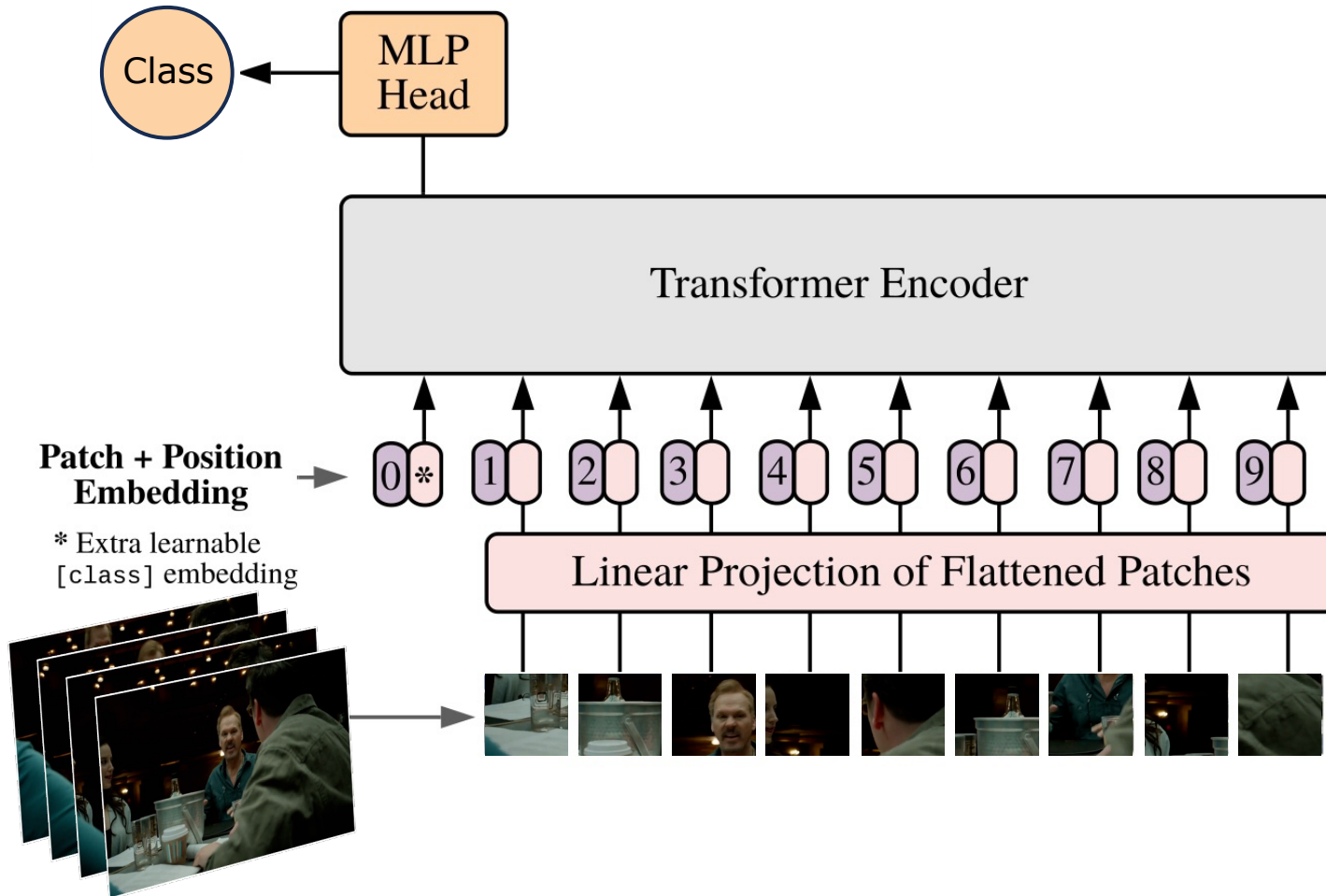


Transformer Encoder



- The image is divided into patches, which are flattened and linearly projected
- A learnable [class] embedding is appended and a Transformer Encoder encodes all tokens
- The transformer [class] token is used for the final classification

TimeSformer



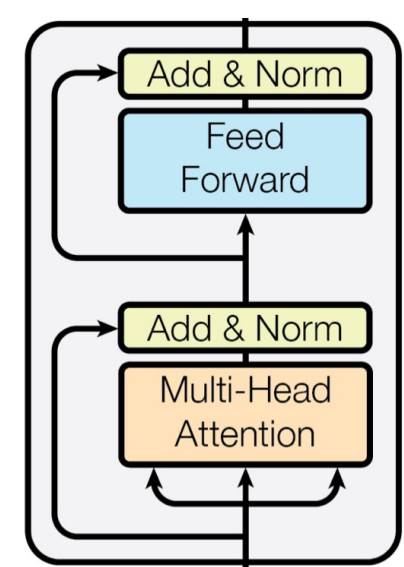
- Same recipe as ViT
- However, spatio-temporal attention is expensive
- Main focus on how to structure the encoder to save computation
- This is done by restricting the attention inside the encoder layers in different ways

G. Bertasius, H. Wang, e L. Torresani, «Is Space-Time Attention All You Need for Video Understanding?», ICML 2021, *arXiv*: arXiv:2102.05095.

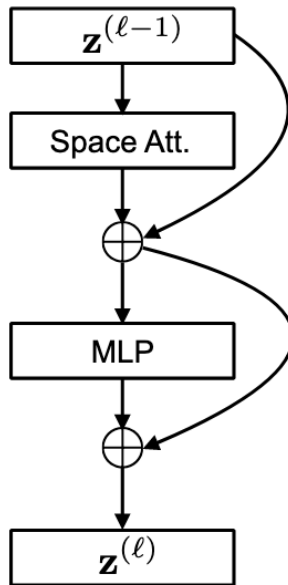
TimeSformer – Self-Attention Block Designs

Video Self-Attention Block Designs

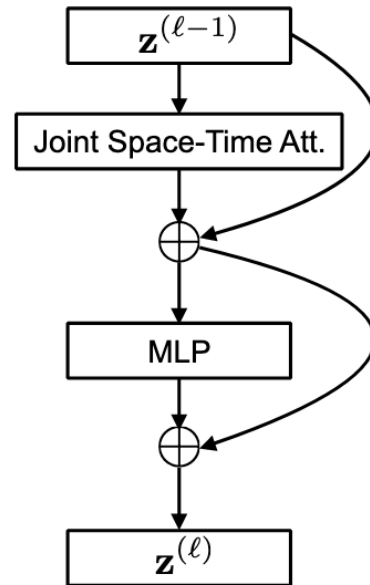
G. Bertasius, H. Wang, e L. Torresani, «Is Space-Time Attention All You Need for Video Understanding?», ICML 2021, *arXiv*: arXiv:2102.05095.



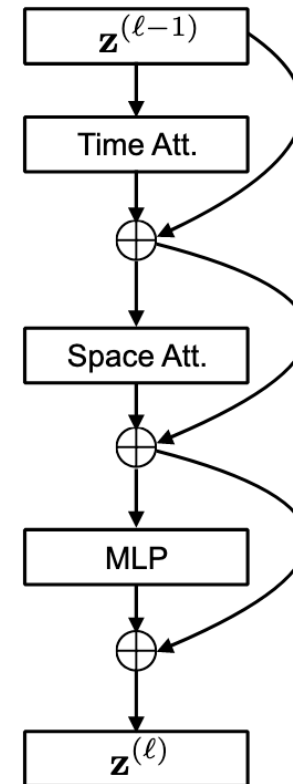
Transformer Self-Attention (for reference)



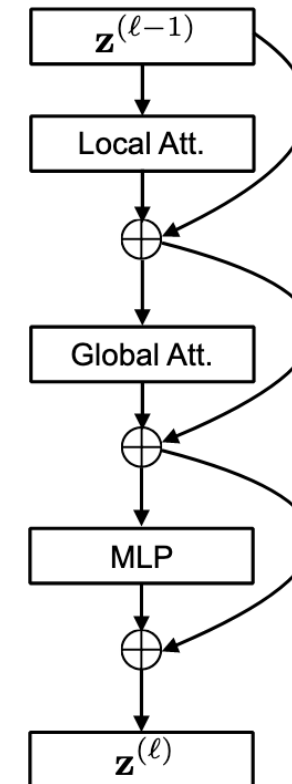
Space Attention (S)



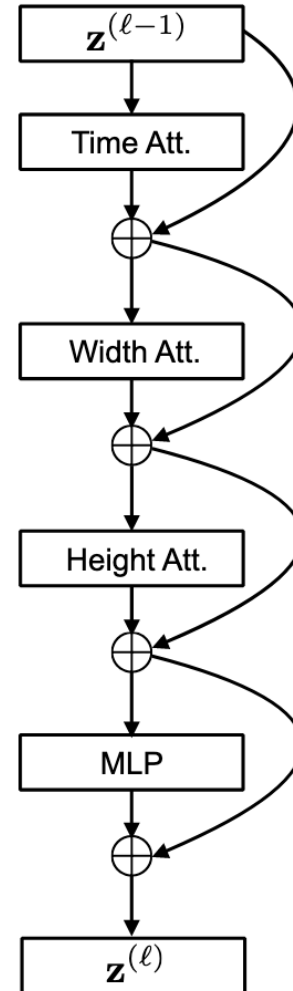
Joint Space-Time Attention (ST)



Divided Space-Time Attention (T+S)



Sparse Local Global Attention (L+G)

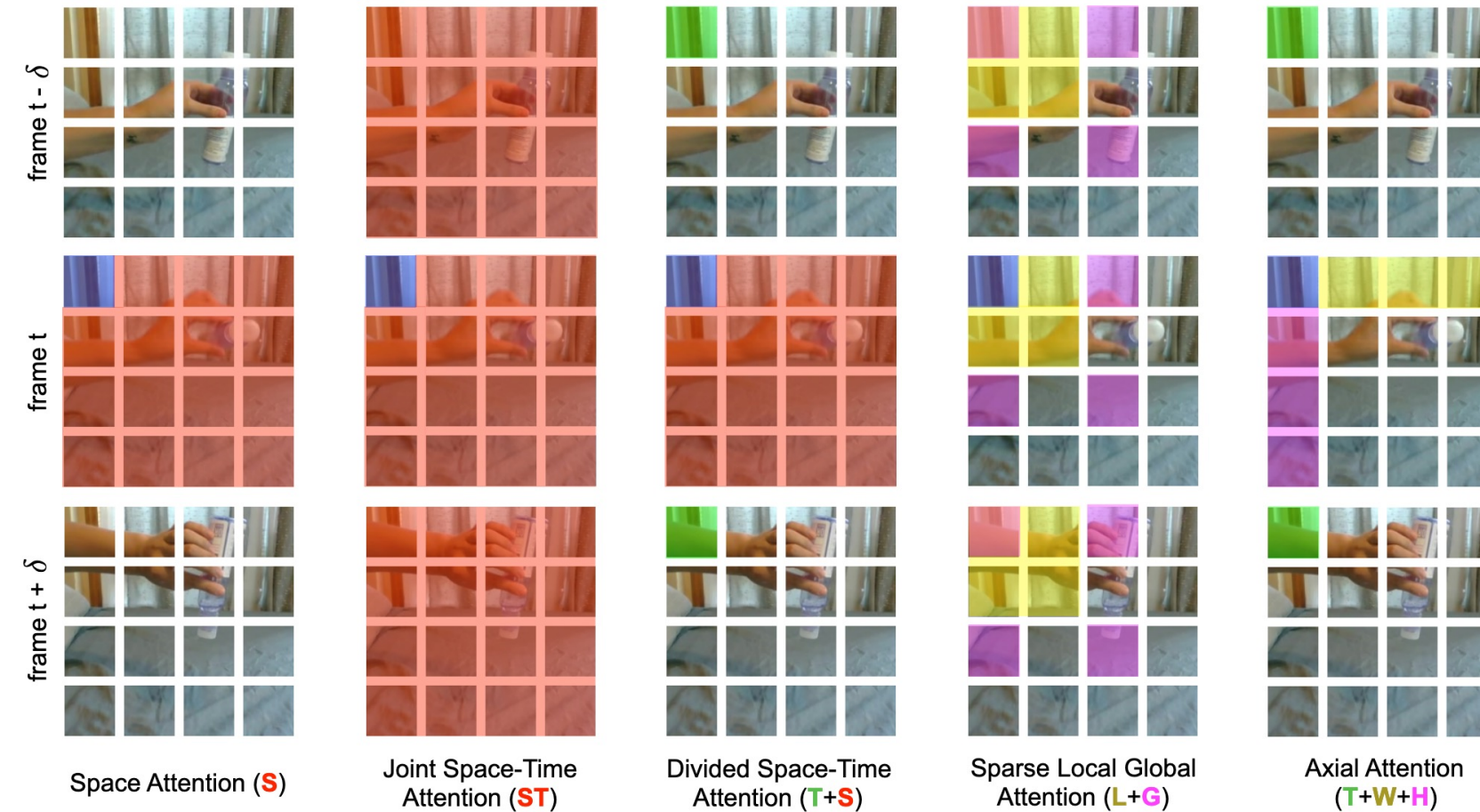


Axial Attention (T+W+H)

TimeSformer - Attention Visualization

G. Bertasius, H. Wang, e L. Torresani, «Is Space-Time Attention All You Need for Video Understanding?», ICML 2021, *arXiv*: arXiv:2102.05095.

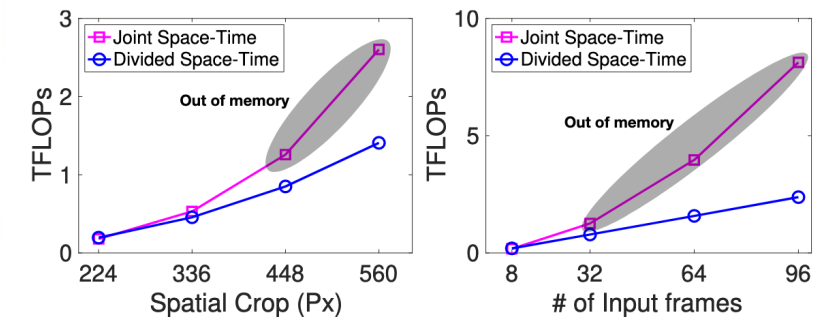
Attention Visualization



Divided attention has more param. and better perf.

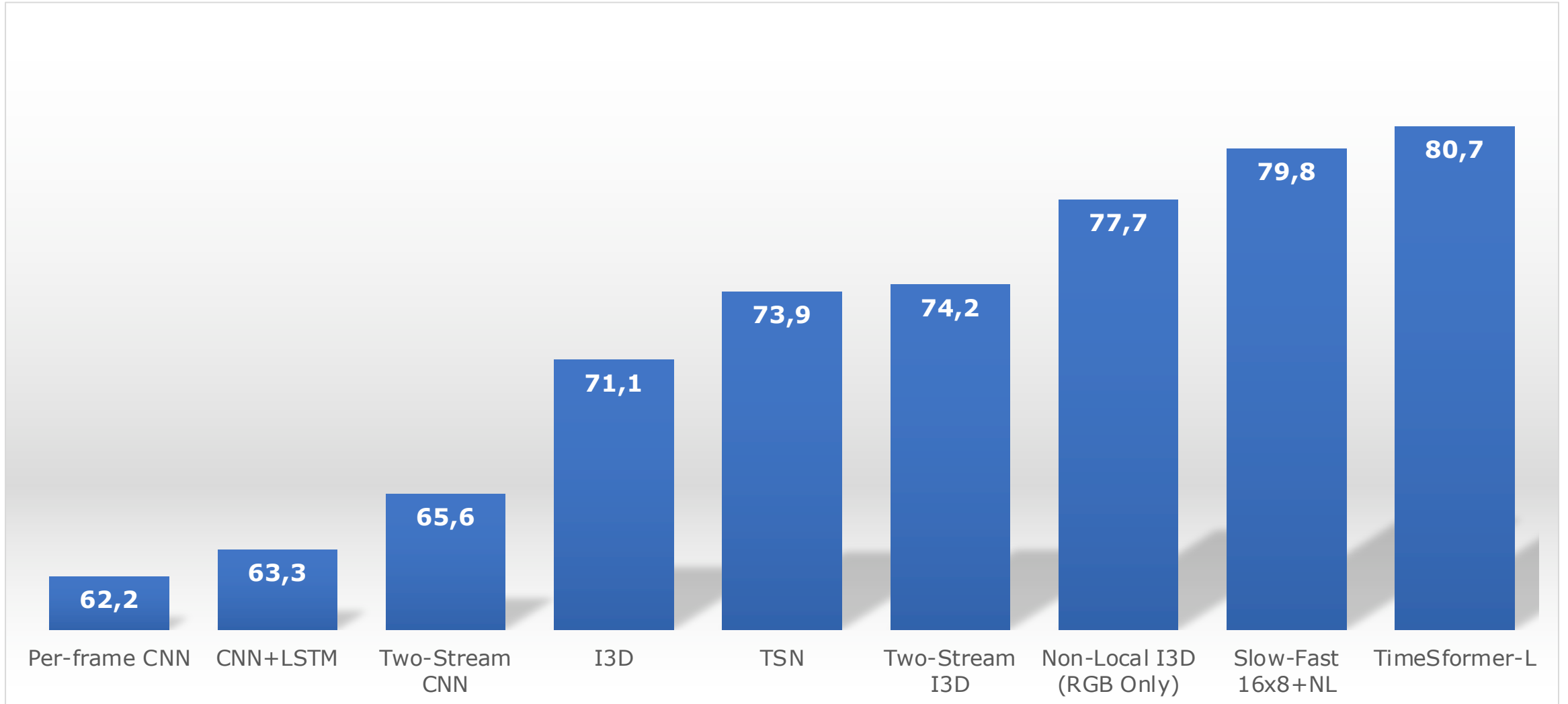
Attention	Params	K400	SSv2
Space	85.9M	76.9	36.6
Joint Space-Time	85.9M	77.4	58.5
Divided Space-Time	121.4M	78.0	59.5
Sparse Local Global	121.4M	75.9	56.3
Axial	156.8M	73.5	56.2

And it's more efficient!



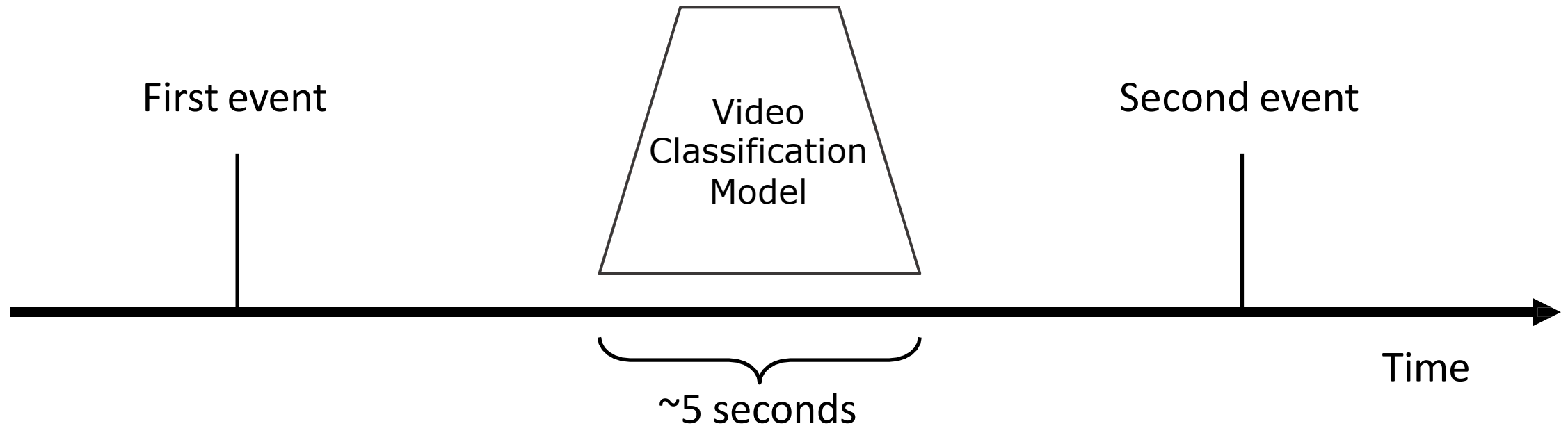
TimeSformer

Top-1 Accuracy on Kinetics-400

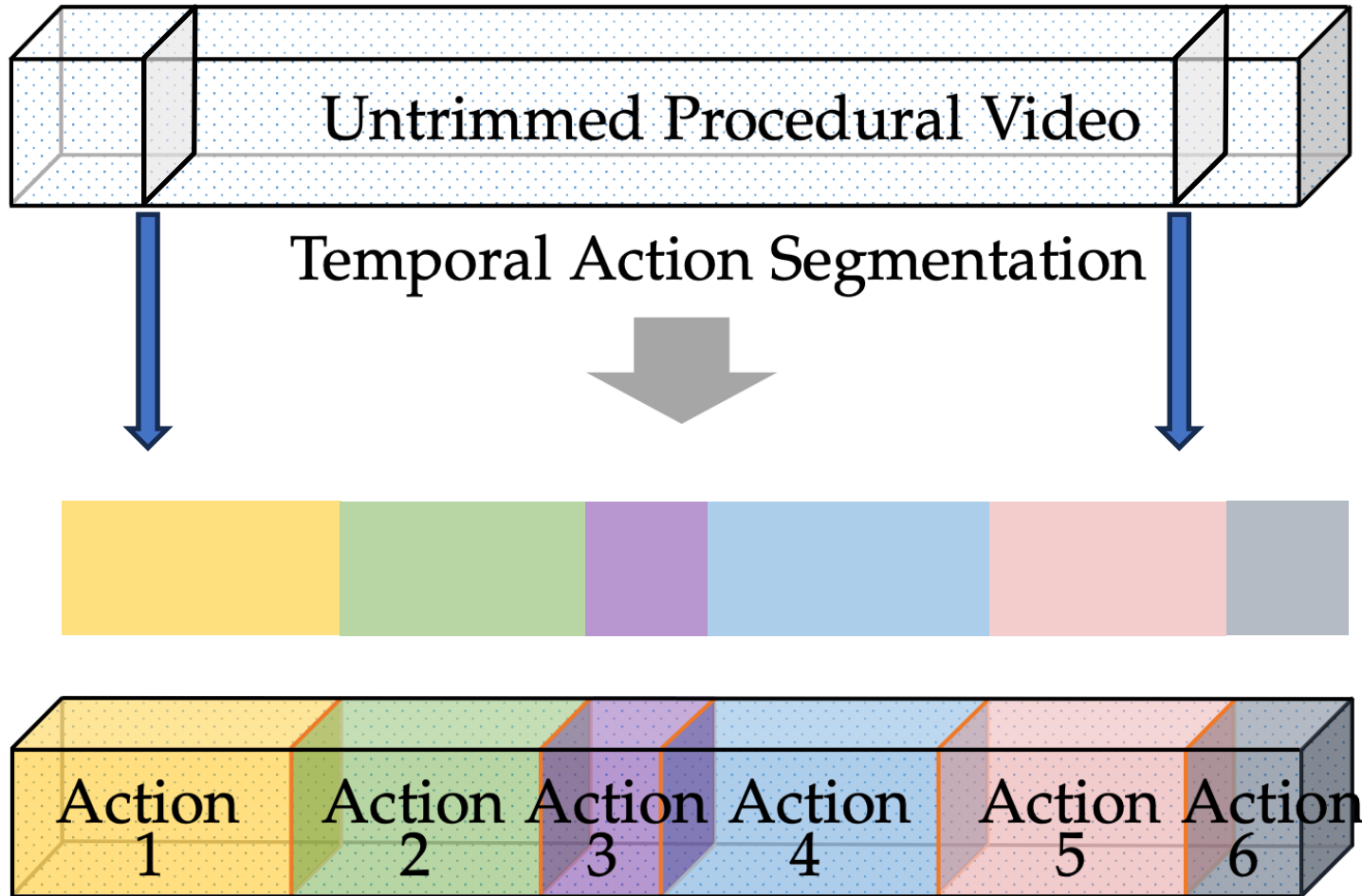


Modeling long-term temporal structure

Video classification models seen so far only model short video clips. What about long-term structure?

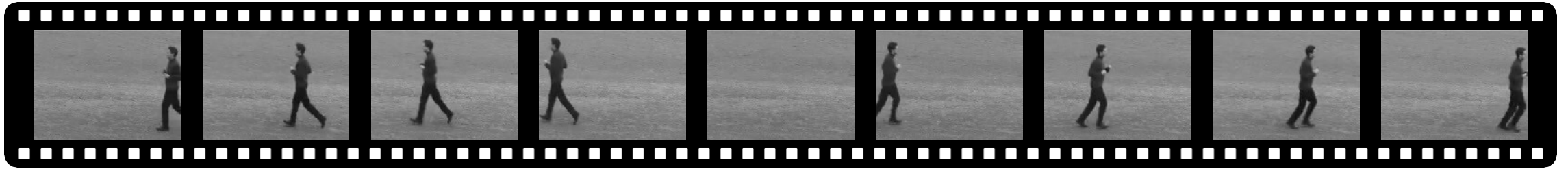


Temporal Action Segmentation



- We'll take temporal action segmentation as an example temporal sequence task
- Temporal action segmentation can be seen as a continuous per-frame action prediction task
- Since actions can span long contexts, predictions cannot be made in isolation
- We need some form of temporal modeling

Different ways to see video



1) Single Image



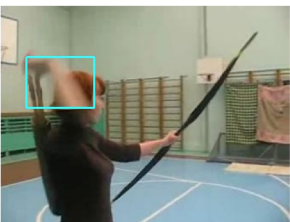
2) A Bag of Images



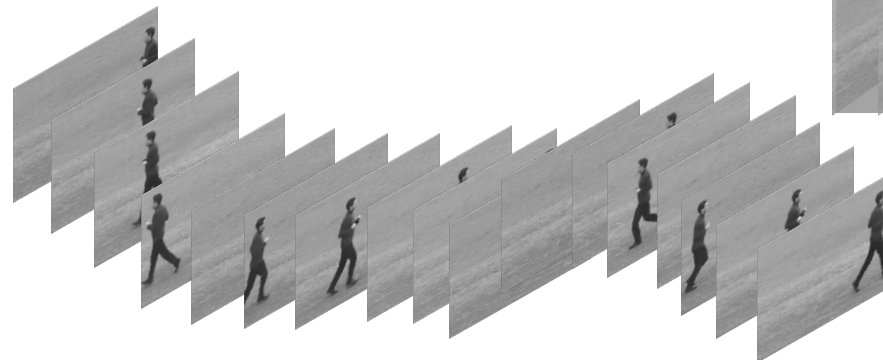
3) 3D (x,y,t) volume



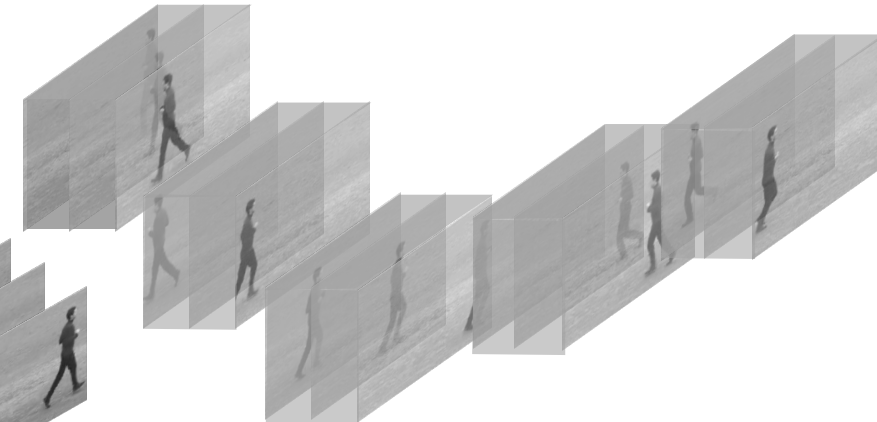
4) Single Image +
Multi-Frame Optical Flow



5) a time-series of images



6) a time-series of (x,y,t) volumens



Long Videos and Feature extraction



Video Model (e.g., pre-trained classifier)



Downstream Model (e.g., action segmentation)

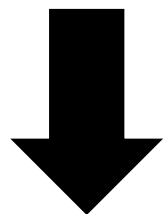
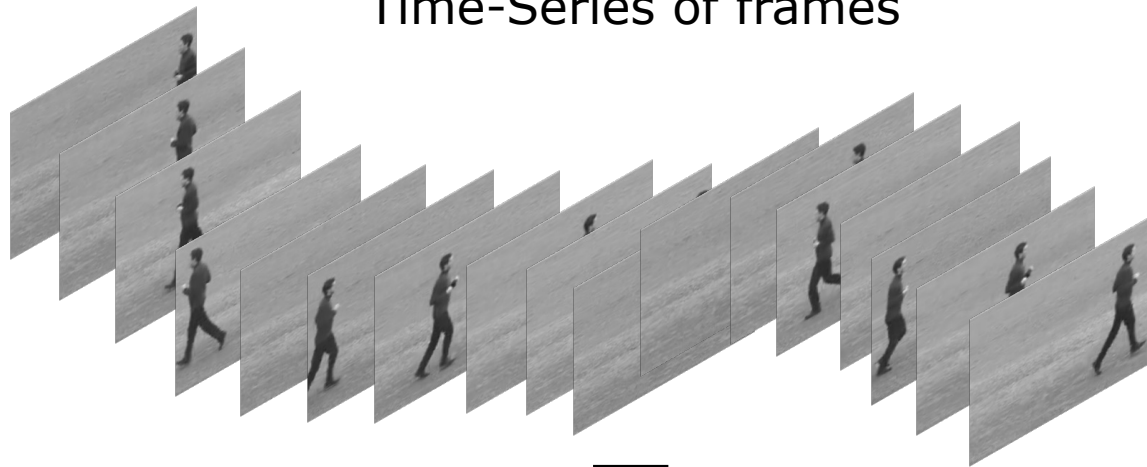


Features can be always batch-extracted at training time and if the task does not require online computation, also at test time.

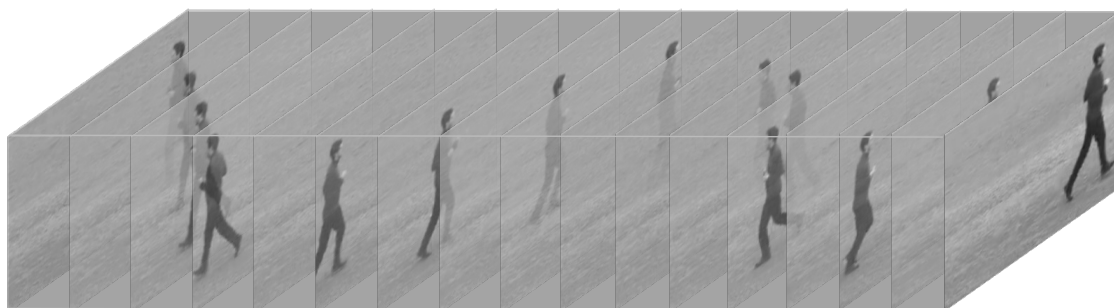
- When video is very long, computation may become very intensive!
- Doing some math, a 30-minutes-long small 224×224 video at 30fps would take about 8GB in memory;
- That's just the video, no model yet!
- A ResNet18 model processing 30 frames would take about 2GB in training (weights, activations, etc.), so that's almost 1TB for 30 minutes!
- In practice, we use a video model to extract features, then train a higher model on higher-level features

From a time-series of frames to a time-series of feature vectors

Time-Series of frames



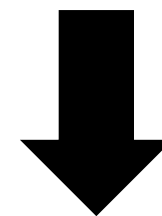
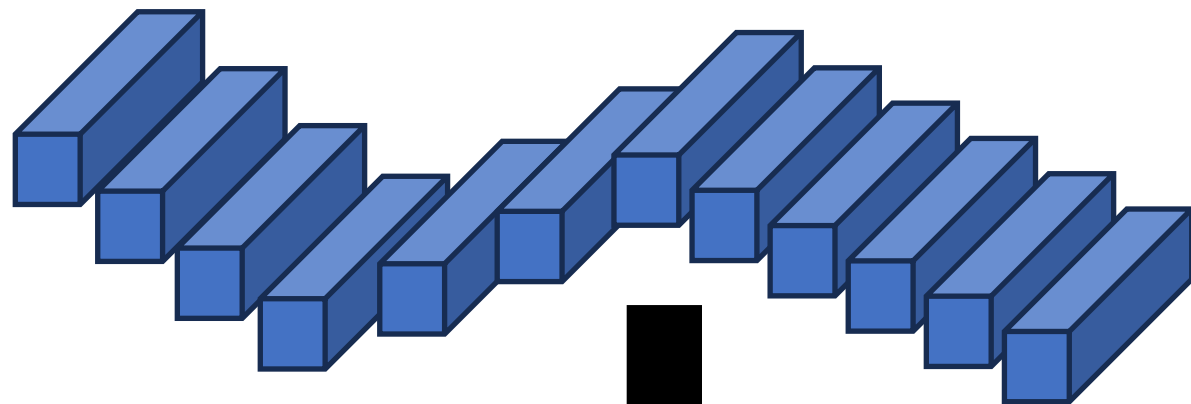
Spatio-Temporal Volume



Uni
ct

$F \times W \times H \times 3$

Time-series of features



Data Matrix

time

$F \times D$

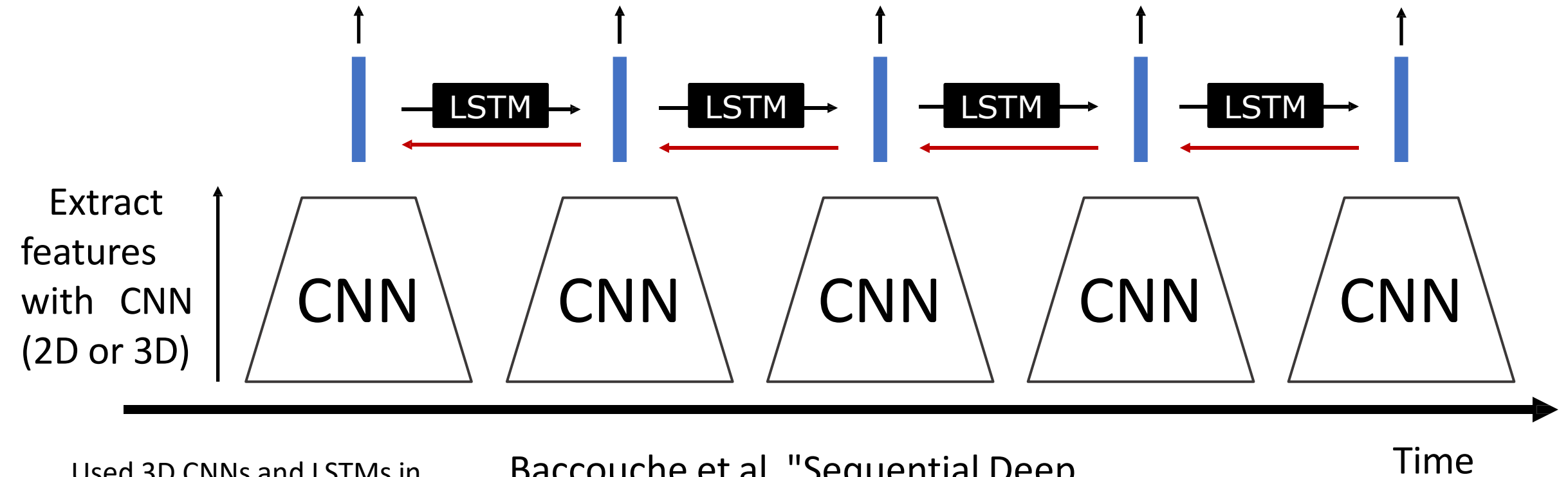
Easier to
manage,
lighter models,
faster to train

Scales well at
inference with
recursive
models

Modeling long-term temporal structure

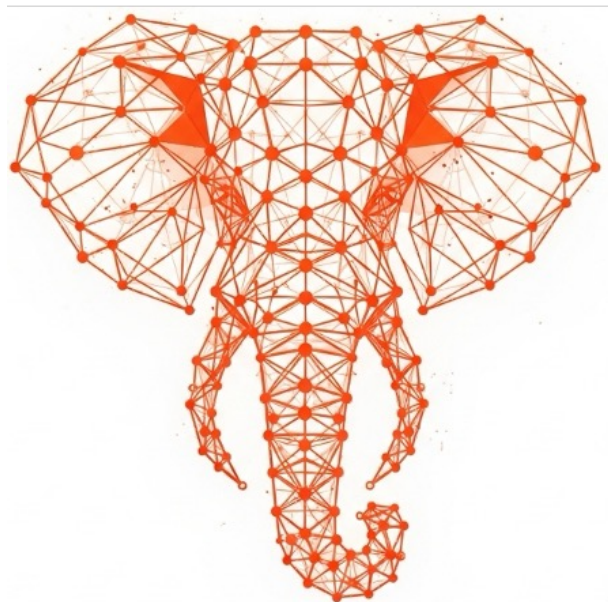
Sometimes don't
backprop to CNN to
save memory; pretrain
and use it as a feature
extractor

Process local features using recurrent network (e.g. LSTM)
Many to many: one output per video frame



Used 3D CNNs and LSTMs in
2011! Way ahead of its time

Baccouche et al, "Sequential Deep
Learning for Human Action
Recognition", **2011**



Uni
ct **DEEP LEARNING**
ADVANCED MODELS AND METHODS

Deep Learning

Advanced Models & Methods

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Laurea Magistrale in Informatica

Dip. di Matematica e Informatica

Università di Catania

The End